Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores 2.
Ing. Mario Tobar.



“MANUAL TÉCNICO - ST4CK OVER-PIG”

Diego José Maldonado Monterroso.
Carné: 201931811.

Quetzaltenango, Guatemala.

Martes 27 de Octubre de 2026.

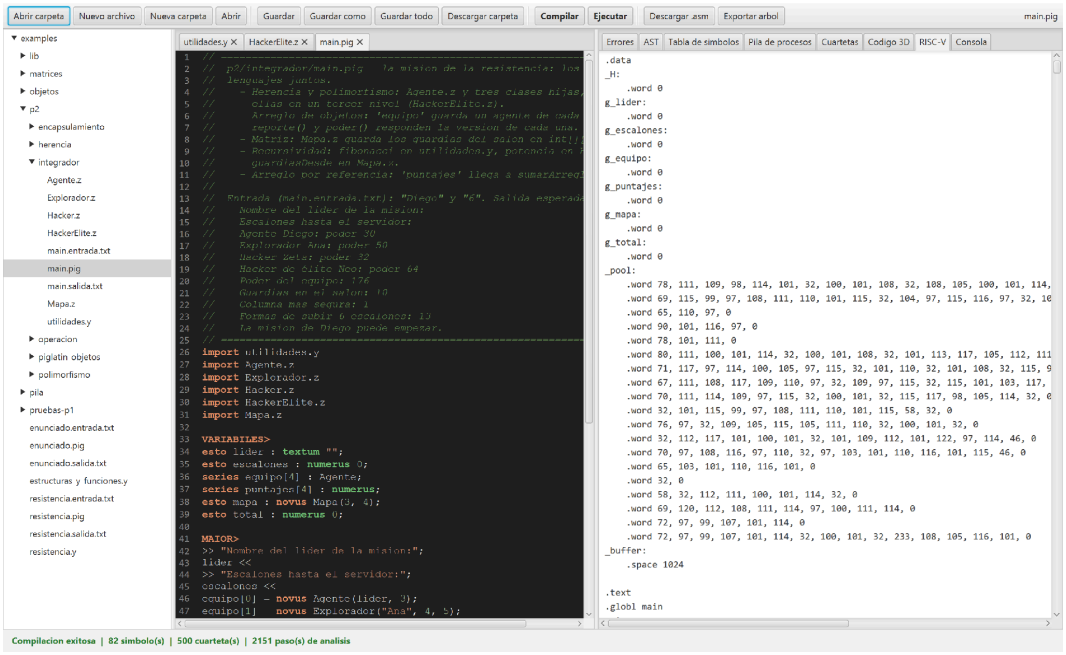
Introducción

Este manual describe cómo está construido **St4ck Over-Pig**: sus tecnologías, su arquitectura, los lenguajes que reconoce, cómo genera el código intermedio y el ensamblador, cómo lo ejecuta y cómo se prueba y se mantiene. Está dirigido a desarrolladores, al catedrático y a la auxiliar del curso; el uso de la aplicación paso a paso está en el manual de usuario (`docs/MANUAL_USUARIO.docx`).

St4ck Over-Pig es un compilador de tres lenguajes de alto nivel a **RISC-V** para el simulador **RARS**:

- **PigLatin** (`.pig`): el programa principal, con sus variables globales, funciones e importaciones.
- **Y?** (`.y`): estructuras y funciones, con bloques por sangría como Python.
- **Zetariano** (`.z`): clases al estilo Java, con encapsulamiento, herencia, polimorfismo y `this`/`super`.

Los tres lenguajes producen un mismo árbol de sintaxis abstracta (AST). Sobre él corren el análisis semántico, la generación de código de tres direcciones (cuartetas) y su traducción a RISC-V. La aplicación ejecuta ese ensamblador **dentro de ella misma**, con el simulador de RARS embebido, y también lo descarga para comprobarlo **fuera**, en RARS por su interfaz o por consola.



La ventana principal después de compilar p2/integrador/main.pig.

Convenciones de este documento: el texto en `monoespaciado` es código, un archivo o un comando. Los nombres de botones y pestañas se escriben como aparecen en la interfaz.

Especificaciones técnicas

Tecnologías y versiones.

Tecnología	Versión	Uso
Java	21 (`maven.compiler.release`); probado con JDK 26.0.2.1	Lenguaje del compilador y de la interfaz.
ANTLR 4	4.13.2 (`antlr4-maven-plugin` y `antlr4-runtime`)	Lexers y parsers de los tres lenguajes; los AST se construyen con los visitors que genera.
JavaFX	21.0.9 (`javafx-controls`)	Interfaz gráfica.
RichTextFX	0.11.3	Área de código con estilos por rango. El color lo decide el lexer propio de cada lenguaje.
RARS	1.6 (licencia MIT), en `lib/`	Simulador RISC-V: la API `rars.api` ejecuta el ensamblador dentro de la aplicación, y el mismo jar sirve como RARS externo.
Maven	3.9 (probado con 3.9.11); `maven-compiler-plugin` 3.8.0; `javafx-maven-plugin` 0.0.8	Construcción, generación de los parsers y ejecución.

Requisitos

- JDK 21 o superior, con `java` en el `PATH` (o `JAVA_HOME` configurado).
- Maven 3.9 o superior.
- Windows 10/11, Linux o macOS: Maven descarga el JavaFX de la plataforma.
- Conexión a Internet en la primera construcción, para que Maven descargue las dependencias. RARS no se descarga: viene en el repositorio.
- Pantalla de 1366 × 768 o mayor. La ventana pide 1440 × 880 y se ajusta sola si no cabe.

Estructura del repositorio

```
st4ck-over-pig/
├── docs/                          MANUAL_TECNICO.docx y MANUAL_USUARIO.docx
├── examples/                      programas de prueba con su .entrada.txt y su .salida.txt
├── lib/rars/rars/1.6/             RARS 1.6 como repositorio Maven del proyecto
├── src/main/antlr4/.../grammar/   PigLexer, PigParser, YLexer, YParser, ZLexer, ZParser (.g4)
├── src/main/java/.../over/pig/    App, controller, model, view y util
├── src/main/resources/st4ck.css   paleta del editor y estilos de la interfaz
├── nbactions.xml                  acciones de NetBeans (clean javafx:run)
└── pom.xml
```

Instalación y construcción

1. Clonar el repositorio:

```
git clone https://github.com/DiegoMaldonado19/st4ck-over-pig.git
```

2. Entrar a la carpeta: `cd st4ck-over-pig`.
3. Compilar y abrir la aplicación: `mvn clean javafx:run`.

Maven genera los parsers durante la fase `generate-sources`, a partir de los `.g4` de `src/main/antlr4`, en `target/generated-sources/antlr4`. No hay que correr ANTLR a mano. En NetBeans basta con abrir la carpeta como proyecto Maven y usar *Run*: `nbactions.xml` corre `clean javafx:run`.

Comandos.

Comando	Qué hace
`mvn -q clean compile`	Compila el proyecto y genera los parsers.
`mvn clean javafx:run`	Compila y abre la aplicación.
`java -jar lib/rars/rars/1.6/rars-1.6.jar`	Abre la interfaz gráfica de RARS, para comprobar un `.asm` por fuera.
`java -Dstdout.encoding=UTF-8 -jar lib/rars/rars/1.6/rars-1.6.jar nc me sm se1 ae2 main.asm < main.entrada.txt`	Ejecuta un `.asm` en RARS por consola, con su entrada: en `cmd`, Git Bash, Linux o macOS; desde PowerShell, dentro de `cmd /c "..."`.

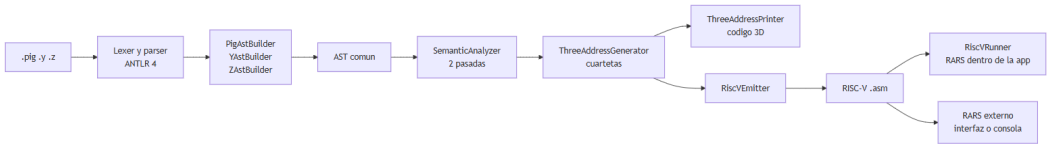
Por qué RARS vive en lib/

RARS no está publicado en Maven Central. Su jar viaja con el repositorio, en `lib/` ordenado como un repositorio Maven (`lib/rars/rars/1.6/rars-1.6.jar` y su `rars-1.6.pom`), que el `pom.xml` declara así:

```
<repository>
    <id>project-lib</id>
    <url>${project.baseUri}lib</url>
    <releases><checksumPolicy>ignore</checksumPolicy></releases>
</repository>
...
<dependency>
    <groupId>rars</groupId>
    <artifactId>rars</artifactId>
    <version>1.6</version>
</dependency>
```

Se probó primero con una dependencia de alcance `system`: compila, pero `javafx:run` arma el classpath de ejecución sin ese jar y la aplicación se cierra con `NoClassDefFoundError: rars/AssemblyException`. Con el repositorio del proyecto la dependencia es normal. `${project.baseUri}` escribe la URL con los espacios ya codificados, y la carpeta del proyecto puede tener espacios. El `.gitignore` excluye los `*.jar`, salvo `!lib/rars/rars/1.6/rars-1.6.jar`.

Arquitectura



El camino de un programa, del archivo fuente a la ejecución.

`CompilerPipeline.compile(source, path)` corre el pipeline completo y devuelve un `CompilationResult`, que es un record. No conoce la interfaz: no imprime ni muestra nada.

1. **Lexer.** Se elige por la extensión del archivo (`Language.fromExtension`); un archivo sin guardar se lee como PigLatin. Los lexers tienen reglas de error propias (`TEXTO_SIN_CERRAR`, `CARACTER_INVALIDO`...). Si aparece una, se reporta como error léxico y el archivo no se parsea.
2. **Parser.** `SyntaxErrorListener` traduce al español los mensajes de ANTLR. En PigLatin, `KeepFinisStrategy` evita que ANTLR descarte el `FINIS` final al recuperarse. Tras un error de sintaxis el análisis sigue, para reportar también los errores semánticos.
3. **Pila de procesos.** `ProcessStackListener` recorre el árbol ya construido y registra cada paso (PUSH, SHIFT, REDUCE, ACCEPT, ERROR) para la pestaña "Pila de procesos".
4. **AST.** `PigAstBuilder`, `YAstBuilder` y `ZAstBuilder` extienden los visitors de ANTLR y producen el mismo AST.
5. **Importaciones.** Cada `import` se resuelve contra la carpeta del `.pig` y se compila con su propio front-end. Los nombres distinguen mayúsculas y minúsculas, como en Linux. Solo el `.pig` importa: tiene que importar todo lo que el programa usa. Un `.z` compilado solo registra las demás clases de su carpeta, como un paquete de Java.
6. **Semántico, en dos pasadas sobre todos los archivos.** `register` registra estructuras, clases y firmas; `visitProgram` revisa los cuerpos. Así el orden de los `import` no importa.

7. **Generación.** Solo si no hay errores y el archivo es un `.pig`: `ThreeAddressGenerator` produce las cuartetas, `ThreeAddressPrinter` las escribe como código 3D y `RiscVEmitter` las traduce a RISC-V.
8. **Ejecución.** El botón "Ejecutar" compila y entrega el RISC-V a `RiscVRunner`, que lo corre en el simulador de RARS en un hilo aparte.

Modelo-vista-controlador

La vista (`view`, JavaFX) no conoce ANTLR, y el modelo (`model`) no conoce JavaFX. `ApplicationController` es el único punto donde se encuentran. Sin ese desacople, cambiar la interfaz o el compilador obligaría a tocar el otro.

Paquetes.

Paquete	Responsabilidad	Clases principales
`(raíz)`	Arranque de JavaFX.	`App`
`controller`	Eventos de la interfaz, archivos y carpetas.	`ApplicationController`, `FileController`
`model`	Pipeline de compilación y ejecución del RISC-V.	`CompilerPipeline`, `RiscVRunner`
`model.analysis`	Parse tree → AST y análisis semántico.	`PigAstBuilder`, `YAstBuilder`, `ZAstBuilder`, `SemanticAnalyzer`, `ReturnPathChecker`
`model.ast`	El AST común: `AstNode`, `AstVisitor<T>` y 40 nodos.	`Program`, `ClassDeclaration`, `FunctionDeclaration`...
`model.codegen`	Cuartetas, código 3D y RISC-V.	`ThreeAddressGenerator`, `ThreeAddressPrinter`, `RiscVEmitter`, `Quadruple`
`model.symbols`	Tabla de símbolos, ámbitos y visibilidad.	`SymbolTable`, `Scope`, `Symbol` y sus subclases, `Visibility`
`model.types`	Tipos y compatibilidad.	`DataType`, `TypeSystem`
`model.errors`	Errores léxicos, sintácticos y semánticos.	`ErrorManager`, `CompilerError`, `SyntaxErrorListener`
`model.stack`	Pila de procesos del parser.	`ProcessStackListener`, `ProcessStep`
`view`	Ventana, editores, coloreado, AST visual y pila.	`MainView`, `EditorTab`, `HighlightingCodeArea`, `ProcessStackPanel`, `AstTreeBuilder`
`util`	Disco y constantes.	`FileManager`, `Constants`

Patrones de diseño.

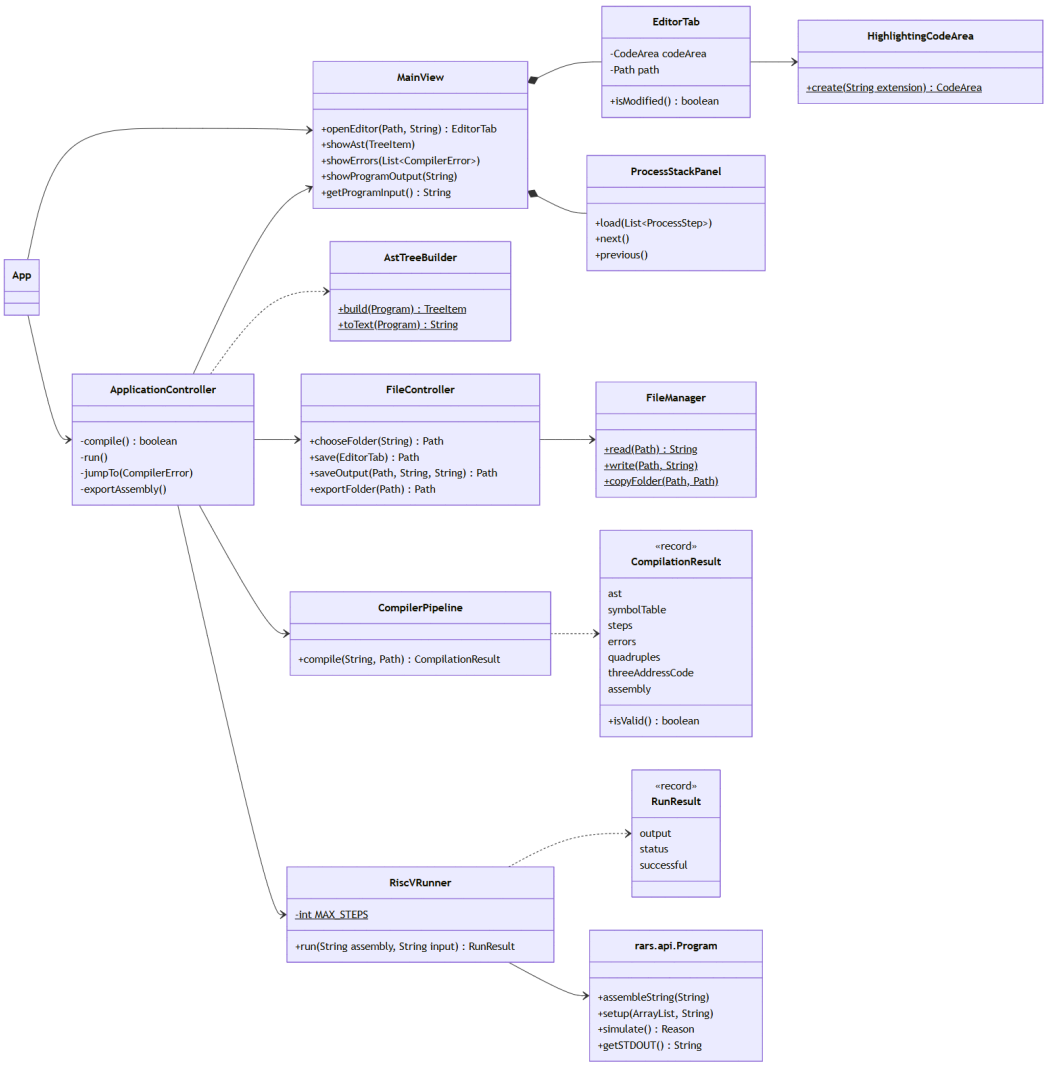
Patrón	Dónde	Para qué
MVC	`view`, `model`, `controller`	La interfaz y el compilador cambian por separado.

Patrón	Dónde	Para qué
Visitor	`AstVisitor<T>`: `SemanticAnalyzer` devuelve el `DataType` y `ThreeAddressGenerator`, el operando	Recorrer el mismo AST con operaciones distintas sin tocar los nodos.
Composite	`AstNode` y sus hijos	Tratar el árbol de forma uniforme: análisis, generación, AST visual y exportación.
Pipeline	`CompilerPipeline`	Etapas independientes que se detienen ante errores.
Observer	`SyntaxErrorListener`, `ProcessStackListener`, eventos de JavaFX	Recibir errores y pasos del parser sin acoplarse a ANTLR.
Strategy	`KeepFinisStrategy` (`DefaultErrorStrategy`)	Cambiar cómo se recupera el parser de PigLatin.
Fábrica estática	`HighlightingCodeArea.create(extension)`	Crear el editor con el lexer de su extensión.
Objeto de datos (record)	`CompilationResult`, `RunResult`, `Quadruple`, `ProcessStep`	Llevar resultados inmutables del modelo a la vista.

Diagrama de clases

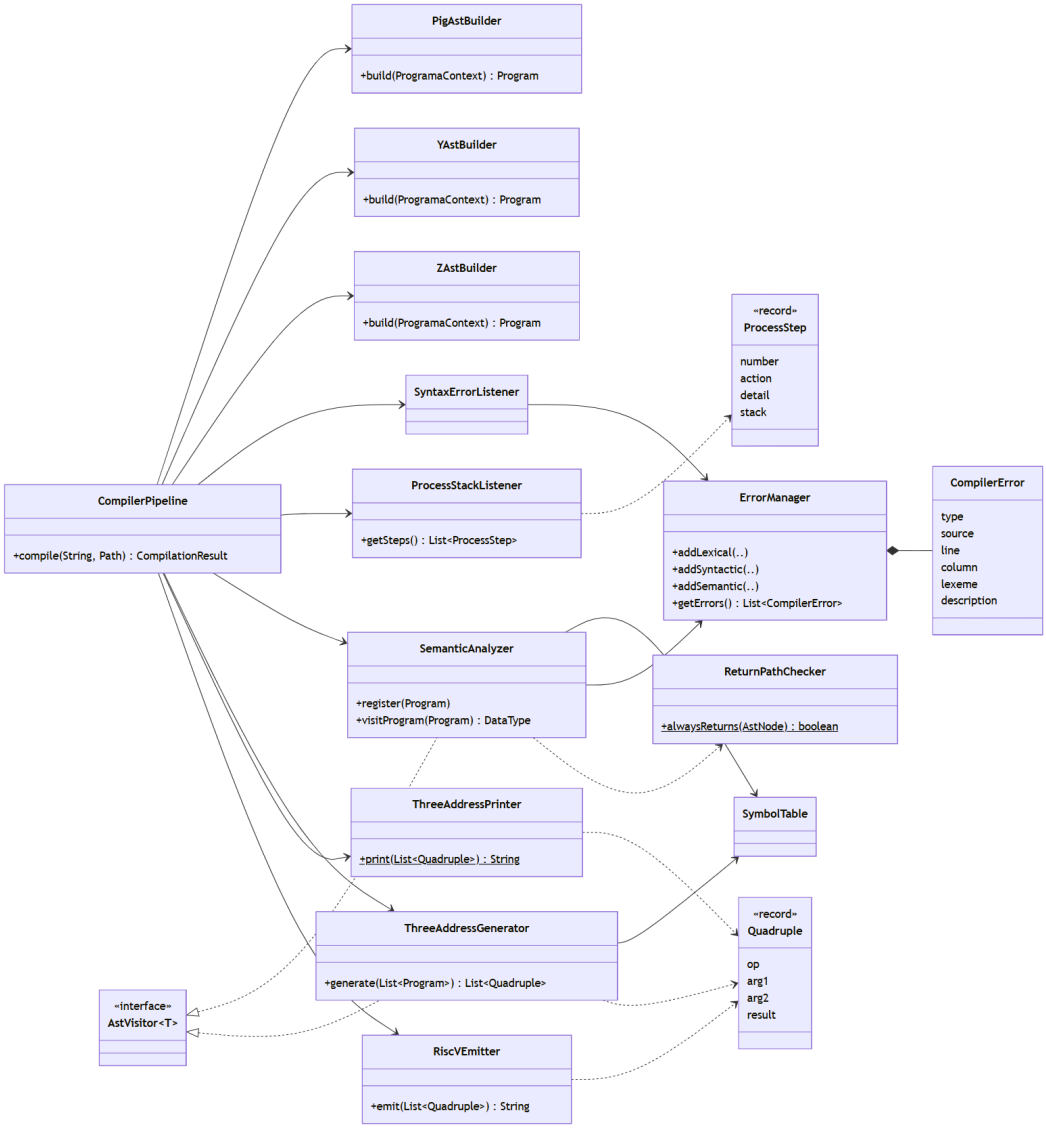
Por legibilidad, el diagrama se divide en cuatro partes. Las flechas sólidas son asociaciones (`-->`), las punteadas, dependencias de uso (`..>`), los rombos, composición y agregación, y los triángulos, herencia o implementación. `$` marca un miembro estático. La fuente Mermaid de la vista general está en el `README.md`.

Interfaz, controlador y ejecución



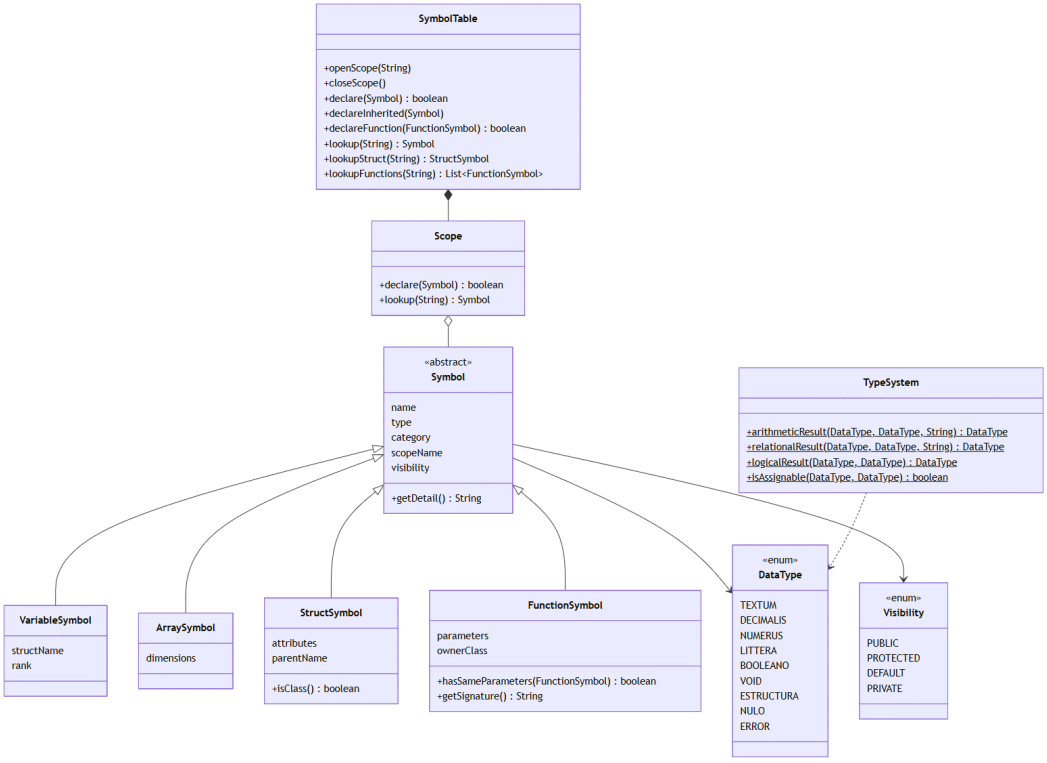
La interfaz, el controlador y las dos entradas al modelo: CompilerPipeline y RiscVRunner.

Análisis y generación de código



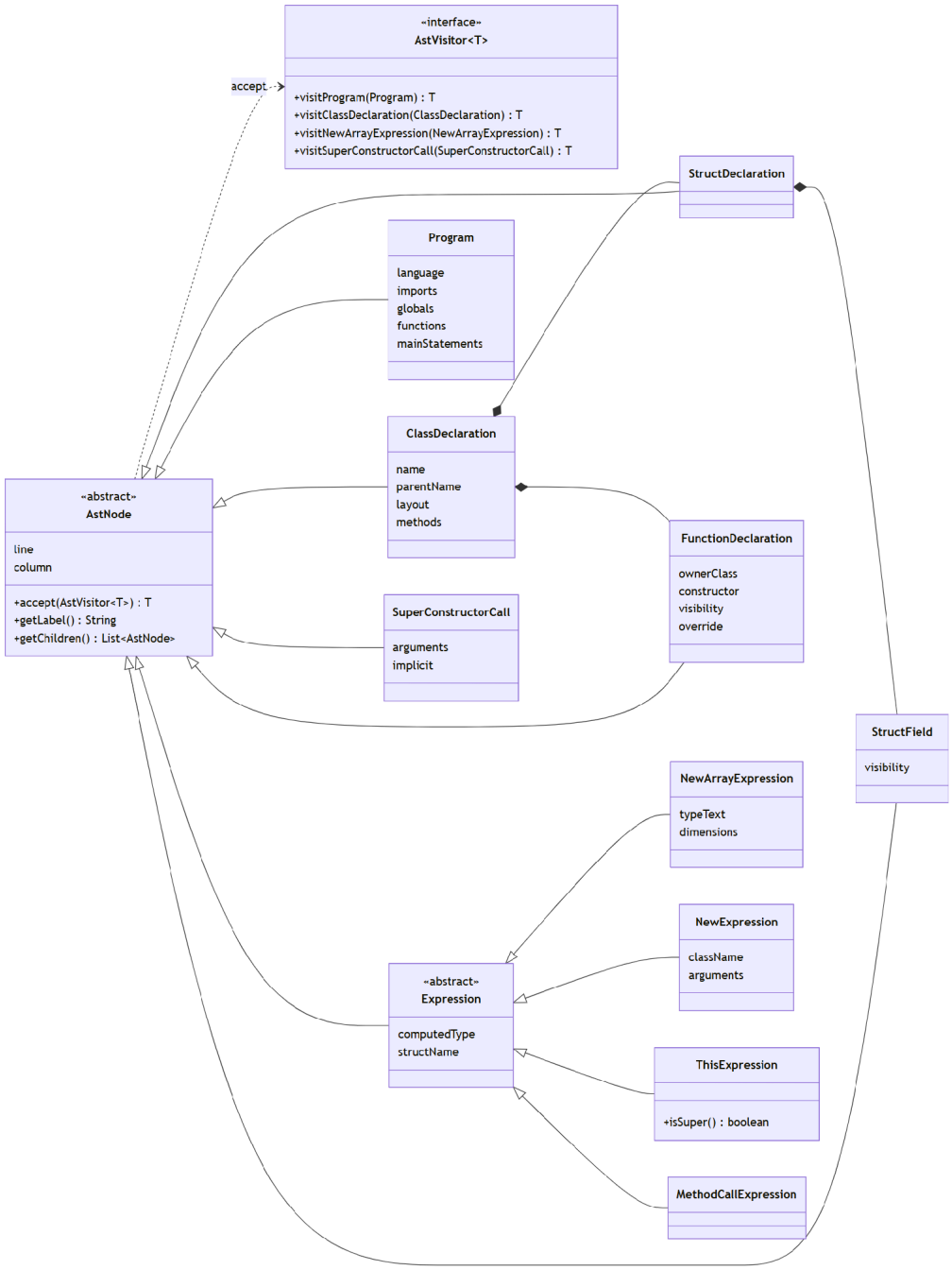
El pipeline, los constructores de AST, el semántico y los tres generadores.

Tabla de símbolos y tipos



La tabla de símbolos, los ámbitos, los cuatro tipos de símbolo, Visibility y el sistema de tipos.

Árbol de sintaxis abstracta



AstNode, Expression y los nodos que agregó el Proyecto 2 o que cambió.

Los 40 nodos del AST.

Grupo	Nodos
Declaraciones (9)	`Program`, `ImportDeclaration`, `ClassDeclaration`, `StructDeclaration`, `StructField`, `FunctionDeclaration`, `Parameter`, `VariableDeclaration`, `ArrayDeclaration`

Grupo	Nodos
Sentencias (16)	`Block`, `Assignment`, `IfStatement`, `WhileStatement`, `DoWhileStatement`, `ForStatement`, `SwitchStatement`, `CaseClause`, `ReturnStatement`, `BreakStatement`, `ContinueStatement`, `PrintStatement`, `InputStatement`, `ExpressionStatement`, `IncrementStatement`, `SuperConstructorCall`
Expresiones (15)	`BinaryExpression`, `UnaryExpression`, `IncrementExpression`, `LiteralExpression`, `IdentifierExpression`, `ArrayAccessExpression`, `MemberAccessExpression`, `FunctionCallExpression`, `MethodCallExpression`, `NewExpression`, `NewArrayExpression`, `ThisExpression`, `TernaryExpression`, `ReadExpression`, `CompositeLiteralExpression`

Clases nuevas del Proyecto 2.

Clase	Paquete	Responsabilidad
`ThreeAddressGenerator`	`model.codegen`	Recorre el AST y produce las cuartetas: código de saltos, registro de activación, heap, despacho polimórfico.
`ThreeAddressPrinter`	`model.codegen`	Escribe las cuartetas como listado de código 3D.
`RiscVEmitter`	`model.codegen`	Traduce cada cuarteta a instrucciones RISC-V para RARS.
`RiscVRunner`	`model`	Ensambla y ejecuta el RISC-V dentro de la aplicación con RARS 1.6.
`NewArrayExpression`	`model.ast.expression`	`new int[f][c]` como expresión, con dimensiones variables.
`SuperConstructorCall`	`model.ast.statement`	`super(...)`, escrito o implícito, al inicio de un constructor.
`Visibility`	`model.symbols`	`PUBLIC`, `PROTECTED`, `DEFAULT` o `PRIVATE` de un miembro.

Palabras reservadas y símbolos

Las tablas de esta sección se generaron leyendo los lexers (`.g4`) del repositorio, así que contienen exactamente lo que el compilador reconoce. Los tres lenguajes distinguen mayúsculas y minúsculas.

PigLatin

Palabras reservadas de PigLatin (PigLexer.g4).

Categoría	Palabras
Marcadores de sección	`VARIABILES> VARIABILES[ MUNERA> MAIOR> FINIS`
Importación	`import`
Declaración	`esto series structura finis`
Tipos y valores lógicos	`numerus decimalis textum littera bool verum falsus`
Control de flujo	`si aliter dum facere per perge interrumpe`
Funciones	`actio ratio reddere`
Objetos	`novus null`
Negación lógica	`non`

Operadores y signos de PigLatin.

Símbolo	Token	Uso
`<<`	ENTRADA	lectura de la consola (`x <<`)
`>>`	SALIDA	impresión en la consola (`>> x`)
`++`	INCREMENTO	incremento
`--`	DECREMENTO	decremento
`==`	IGUALDAD	igual a
`!=`	DIFERENTE	distinto de
`<=`	MENOR_IGUAL	menor o igual
`>=`	MAYOR_IGUAL	mayor o igual
`&&`	AND	y lógico (corto circuito)
`||`	OR	o lógico (corto circuito)
`+`	MAS	suma o concatenación
`-`	MENOS	resta o signo negativo
`*`	POR	producto
`/`	DIVISION	división (entera entre enteros)
`<`	MENOR	menor que
`>`	MAYOR	mayor que

Símbolo	Token	Uso
`=`	ASIGNACION	asignación
`(`	PAR_IZQ	abre agrupación, parámetros o argumentos
`)`	PAR_DER	cierra
`{`	LLAVE_IZQ	abre bloque o literal
`}`	LLAVE_DER	cierra
`[`	COR_IZQ	abre índice o dimensión
`]`	COR_DER	cierra
`;`	PUNTO_COMA	fin de instrucción
`:`	DOS_PUNTOS	tipo, caso o bloque
`,`	COMA	separador
`.`	PUNTO	acceso a atributo o método, y ruta de un import

Y?

Palabras reservadas de Y? (YLexer.g4).

Categoría	Palabras
Marcadores de sección	`%estructuras %funciones`
Declaración	`estructura definir retornar`
Tipos y valores lógicos	`entero cadena flotante caracter bool verdadero falso`
Control de flujo	`si entonces sino contrario elegir caso siempre romper continuar para mientras hacer`
Funciones del sistema	`imprimir leer`

Operadores y signos de Y?.

Símbolo	Token	Uso
`->`	FLECHA	tipo de retorno de una función
`++`	INCREMENTO	incremento
`--`	DECREMENTO	decremento
`==`	IGUALDAD	igual a
`!=`	DIFERENTE	distinto de
`<=`	MENOR_IGUAL	menor o igual
`>=`	MAYOR_IGUAL	mayor o igual
`&&`	AND	y lógico (corto circuito)
`||`	OR	o lógico (corto circuito)
`+`	MAS	suma o concatenación
`-`	MENOS	resta o signo negativo

Símbolo	Token	Uso
`*`	POR	producto
`/`	DIVISION	división (entera entre enteros)
`<`	MENOR	menor que
`>`	MAYOR	mayor que
`=`	ASIGNACION	asignación
`!`	NEGACION	negación lógica
`(`	PAR_IZQ	abre agrupación, parámetros o argumentos
`)`	PAR_DER	cierra
`{`	LLAVE_IZQ	abre un literal o el cuerpo de un elegir
`}`	LLAVE_DER	cierra
`[`	COR_IZQ	abre índice o dimensión
`]`	COR_DER	cierra
`;`	PUNTO_COMA	fin de instrucción
`:`	DOS_PUNTOS	tipo, caso o bloque
`,`	COMA	separador
`.`	PUNTO	acceso a un campo de una estructura

Además, `YLexer` fabrica tres tokens que no tienen texto: `INDENT` y `DEDENT`, al abrir y cerrar un bloque por sangría, y `SANGRIA_INVALIDA`, cuando una línea vuelve a una sangría que ningún bloque abierto tiene. El tabulador avanza hasta el siguiente múltiplo de 8, como en Python. Dentro de `( )`, `[ ]` o un literal `{ }` la sangría no cuenta.

Zetariano

Palabras reservadas de Zetariano (ZLexer.g4).

Categoría	Palabras
Encapsulamiento	`public private protected`
Estructura del programa	`class void return new this`
Herencia y polimorfismo	`extends super @Override`
Tipos y valores lógicos	`int double char boolean String`
Literales con nombre	`true false null`
Control de flujo	`if else switch case default break continue for while do`
Funciones del sistema	`println print readln`

Operadores y signos de Zetariano.

Símbolo	Token	Uso
`++`	INCREMENTO	incremento
`--`	DECREMENTO	decremento
`+=`	MAS_IGUAL	suma y asignación
`-=`	MENOS_IGUAL	resta y asignación
`*=`	POR_IGUAL	producto y asignación
`==`	IGUALDAD	igual a
`!=`	DIFERENTE	distinto de
`<=`	MENOR_IGUAL	menor o igual
`>=`	MAYOR_IGUAL	mayor o igual
`&&`	AND	y lógico (corto circuito)
`||`	OR	o lógico (corto circuito)
`+`	MAS	suma o concatenación
`-`	MENOS	resta o signo negativo
`*`	POR	producto
`/`	DIVISION	división (entera entre enteros)
`%`	MODULO	residuo entero
`<`	MENOR	menor que
`>`	MAYOR	mayor que
`=`	ASIGNACION	asignación
`!`	NEGACION	negación lógica
`?`	INTERROGA	operador ternario `c ? a : b`
`(`	PAR_IZQ	abre agrupación, parámetros o argumentos
`)`	PAR_DER	cierra
`{`	LLAVE_IZQ	abre bloque o literal de arreglo
`}`	LLAVE_DER	cierra
`[`	COR_IZQ	abre índice o dimensión
`]`	COR_DER	cierra
`;`	PUNTO_COMA	fin de instrucción
`:`	DOS_PUNTOS	tipo, caso o bloque
`,`	COMA	separador
`.`	PUNTO	acceso a atributo o método

Literales, identificadores y comentarios

Tokens comunes a los tres lenguajes.

Token	Forma	Ejemplos
`ENTERO`	uno o más dígitos	`0`, `2147483647`
`DECIMAL`	dígitos, punto y dígitos (en Y? los decimales son opcionales)	`9.81`, `36.` (Y?)
`TEXTO`	entre comillas dobles; escapes `\n \t \r \" \' \\`	`"Hola\n"`
`CARACTER`	un símbolo, o un escape, entre comillas simples	`'a'`, `'\n'`
`ID`	letra o _, seguida de letras, dígitos o _; las letras incluyen á é í ó ú ñ ü	`total`, `año_2`
Comentarios	`// línea` y `/* bloque */`; PigLatin también `## bloque ##`	`// nota`

Errores léxicos: los reconocen reglas propias de cada lexer, con estos mensajes.

Token de error	Mensaje
`TEXTO_SIN_CERRAR`	Cadena sin cerrar: falta la comilla doble final.
`CARACTER_SIN_CERRAR`	Caracter sin cerrar: falta la comilla simple final.
`COMENTARIO_SIN_CERRAR`	Comentario de bloque sin cerrar: falta '*/'.
`COMENTARIO_HASH_SIN_CERRAR` (PigLatin)	Comentario de bloque sin cerrar: falta '##'.
`CARACTER_LARGO`	Un caracter lleva un solo simbolo: un texto va entre comillas dobles.
`CARACTER_MAL_FORMADO`	Caracter vacio o escape invalido: validos `\n \t \r \' \" \\`.
`SANGRIA_INVALIDA` (Y?)	La sangria no coincide con la de ningun bloque abierto.
`CARACTER_INVALIDO`	Simbolo no reconocido por el lenguaje.

Gramáticas

Las gramáticas están escritas para ANTLR 4 y no usan recursividad por la izquierda: la precedencia se codifica con una regla por nivel, de menor a mayor. En Zetariano el ternario va por debajo de `||`; PigLatin no tiene `%` ni ternario. Las etiquetas `# nombre` dan nombre a cada alternativa: ANTLR genera un método `visitNombre` para cada una, y el constructor del AST de cada lenguaje los implementa.

Precedencia de los operadores, de menor a mayor.

Nivel	Operadores	Regla
1	`c ? a : b` (solo Zetariano)	`expresionTernaria`
2	`||`	`expresionOr`
3	`&&`	`expresionAnd`
4	`== !=`	`expresionIgualdad`
5	`< > <= >=`	`expresionRelacional`
6	`+ -`	`expresionAditiva`
7	`* / %` (`%` solo Zetariano)	`expresionMultiplicativa`
8	`non` / `!`, `-`, `++` y `--` prefijos	`expresionUnaria`
9	`[i]`, `.atributo`, `.metodo(..)`, `++` y `--` sufijos	`expresionSufijo`

PigLatin (PigParser.g4)

```
parser grammar PigParser;

options { tokenVocab = PigLexer; }

/* ========================================================================
 * 1. ESTRUCTURA GLOBAL DEL PROGRAMA
 * ======================================================================== */
programa
    : seccionImportaciones? seccionVariables? seccionFunciones? seccionPrincipal
      FIN_PROGRAMA PUNTO_COMA EOF
    ;

// import carpeta.Objeto1.z  -- el ultimo ID es la extension del archivo
seccionImportaciones : importacion+ ;
importacion          : IMPORT ID ( PUNTO ID )+ PUNTO_COMA? ;

seccionVariables : VARIABILES declaracionGlobal* ;
seccionFunciones : MUNERA declaracionFuncion* ;
seccionPrincipal : MAIOR instruccion* ;

// declaracionEstructura sigue aqui aunque el enunciado la prohibe en .pig: el
// semantico la rechaza con un mensaje legible en vez de un error sintactico crudo.
declaracionGlobal
    : declaracionVariable
    | declaracionArreglo
    | declaracionEstructura
    ;

/* ========================================================================
 * 2. DECLARACIONES
```

```
 * ========================================================================== */

// esto p : novus Profesor(..) : Persona;  el tipo explicito es el de la variable,
// y el novus puede crear una clase hija suya. esto p : null : Persona;  un
// objeto que todavia no existe.
declaracionVariable
    : ESTO ID DOS_PUNTOS tipo ASIGNACION? literalCompuesto PUNTO_COMA?  #
      declaracionVariableEstructura
    | ESTO ID DOS_PUNTOS nuevoObjeto ( DOS_PUNTOS tipo )? PUNTO_COMA    # declaracionVariableObjeto
    | ESTO ID DOS_PUNTOS NULO DOS_PUNTOS tipo PUNTO_COMA                # declaracionVariableNula
    | ESTO ID DOS_PUNTOS tipo ( ASIGNACION? expresion )? PUNTO_COMA     # declaracionVariableSimple
    ;

declaracionArreglo
    : SERIES ID ( COR_IZQ expresion COR_DER )+ DOS_PUNTOS tipo?
      ( LLAVE_IZQ listaExpresiones? LLAVE_DER )?
      PUNTO_COMA
    ;

declaracionEstructura
    : STRUCTURA ID LLAVE_IZQ atributoEstructura* LLAVE_DER FINIS PUNTO_COMA
    ;

atributoEstructura
    : ESTO   ID DOS_PUNTOS tipo ( PUNTO_COMA | COMA )?   # campoSimple
    | SERIES ID DOS_PUNTOS tipo ( PUNTO_COMA | COMA )?   # campoArreglo
    ;

tipo
    : NUMERUS
    | DECIMALIS
    | TEXTUM
    | LITTERA
    | BOOL
    | VERUM
    | FALSUS
    | ID        // tipo definido por el usuario (structura)
    ;

/* ========================================================================
 * 3. FUNCIONES
 * ========================================================================== */
declaracionFuncion
    : funcionSinRetorno
    | funcionConRetorno
    ;

funcionSinRetorno
    : ACTIO ID PAR_IZQ listaParametros? PAR_DER cuerpoFuncion FINIS PUNTO_COMA
    ;

funcionConRetorno
    : RATIO tipo ID PAR_IZQ listaParametros? PAR_DER cuerpoFuncion FINIS PUNTO_COMA
    ;

cuerpoFuncion
    : LLAVE_IZQ seccionVariablesLocales? instruccion* LLAVE_DER
    ;

seccionVariablesLocales
    : VARIABILES_LOCAL declaracionGlobal* COR_DER
    ;

listaParametros : parametro ( COMA parametro )* ;
parametro       : ESTO ID DOS_PUNTOS tipo ;
bloque          : LLAVE_IZQ instruccion* LLAVE_DER ;

/* ========================================================================
 * 4. INSTRUCCIONES
 * ========================================================================== */
instruccion
    : declaracionVariable
    | declaracionArreglo
    | declaracionEstructura
    | instruccionSi
    | instruccionMientras
```

```
    | instruccionHacerMientras
    | instruccionPara
    | instruccionRetorno
    | instruccionInterrumpe
    | instruccionPerge
    | instruccionSalida
    | instruccionEntrada
    | asignacion
    | instruccionIncremento
    | instruccionExpresion   // ultima: "x++;" tambien es expresion, y gana instruccionIncremento
    ;

asignacion
    : destino ASIGNACION literalCompuesto PUNTO_COMA?  # asignacionEstructura
    | destino ASIGNACION expresion PUNTO_COMA          # asignacionSimple
    ;

destino : ID sufijoDestino* ;

sufijoDestino
    : COR_IZQ expresion COR_DER   # accesoIndice
    | PUNTO ID                    # accesoAtributo
    ;

instruccionIncremento : destino ( INCREMENTO | DECREMENTO ) PUNTO_COMA ;

// Cualquier expresion, para que un "pergue;" llegue al semantico en vez de
// abortar como error sintactico: el semantico decide que puede ir suelto.
instruccionExpresion : expresion PUNTO_COMA ;

instruccionSi
    : SI PAR_IZQ expresion PAR_DER bloque
      aliterCondicional*
      ( ALITER bloque )?
      FINIS PUNTO_COMA
    ;

aliterCondicional : ALITER PAR_IZQ expresion PAR_DER bloque ;

instruccionMientras
    : DUM PAR_IZQ expresion PAR_DER bloque FINIS PUNTO_COMA
    ;

instruccionHacerMientras
    : FACERE bloque DUM PAR_IZQ expresion PAR_DER PUNTO_COMA
    ;

instruccionPara
    : PER PAR_IZQ inicializacionPara expresion PUNTO_COMA
      actualizacionPara PAR_DER bloque ( FINIS PUNTO_COMA )?
    ;

inicializacionPara : declaracionVariable | asignacion ;

actualizacionPara
    : destino ( INCREMENTO | DECREMENTO )   # actualizacionUnaria
    | destino ASIGNACION expresion          # actualizacionAsignacion
    ;

instruccionRetorno    : REDDERE expresion? PUNTO_COMA ;
instruccionInterrumpe : INTERRUMPE PUNTO_COMA ;
instruccionPerge      : PERGE PUNTO_COMA ;

instruccionSalida : SALIDA expresion ( SALIDA expresion )* PUNTO_COMA ;

instruccionEntrada : destino? ENTRADA PUNTO_COMA? ;

/* ================================================================
 * 5. EXPRESIONES  -- SIN RECURSIVIDAD POR LA IZQUIERDA --
 * ================================================================ */
expresion : expresionOr ;

expresionOr             : expresionAnd ( OR expresionAnd )* ;
expresionAnd            : expresionIgualdad ( AND expresionIgualdad )* ;
expresionIgualdad       : expresionRelacional ( ( IGUALDAD | DIFERENTE ) expresionRelacional )* ;
expresionRelacional     : expresionAditiva ( ( MENOR | MAYOR | MENOR_IGUAL | MAYOR_IGUAL )
```

```
                          expresionAditiva )* ;
expresionAditiva        : expresionMultiplicativa ( ( MAS | MENOS ) expresionMultiplicativa )* ;
expresionMultiplicativa : expresionUnaria ( ( POR | DIVISION ) expresionUnaria )* ;

expresionUnaria
    : NON expresionUnaria                           # unariaNegacionLogica
    | MENOS expresionUnaria                         # unariaNegativo
    | ( INCREMENTO | DECREMENTO ) expresionUnaria   # unariaPrefija
    | expresionSufijo                               # unariaSufijoDelegado
    ;

expresionSufijo : expresionPrimaria sufijoExpresion* ;

sufijoExpresion
    : COR_IZQ expresion COR_DER                   # sufijoIndice
    | PUNTO ID PAR_IZQ listaExpresiones? PAR_DER  # sufijoMetodo
    | PUNTO ID                                    # sufijoAtributo
    | INCREMENTO                                  # sufijoIncremento
    | DECREMENTO                                  # sufijoDecremento
    ;

expresionPrimaria
    : ENTERO                      # primariaEntero
    | DECIMAL                     # primariaDecimal
    | TEXTO                       # primariaTexto
    | CARACTER                    # primariaCaracter
    | VERUM                       # primariaVerdadero
    | FALSUS                      # primariaFalso
    | NULO                        # primariaNulo
    | llamadaFuncion              # primariaLlamada
    | nuevoObjeto                 # primariaNovus
    | ID                          # primariaIdentificador
    | PAR_IZQ expresion PAR_DER   # primariaAgrupacion
    | literalCompuesto            # primariaLiteral
    ;

literalCompuesto
    : LLAVE_IZQ campoLiteral ( ( COMA | PUNTO_COMA ) campoLiteral )* LLAVE_DER  # literalConNombre
    | LLAVE_IZQ listaExpresiones? LLAVE_DER                                     # literalPosicional
    ;

campoLiteral : ID DOS_PUNTOS expresion ;

llamadaFuncion   : ID PAR_IZQ listaExpresiones? PAR_DER ;
nuevoObjeto      : NOVUS ID PAR_IZQ listaExpresiones? PAR_DER ;   // clase de un archivo .z
listaExpresiones : expresion ( COMA expresion )* ;
```

Y? (YParser.g4)

El parser de Y? no ve saltos de línea: los convierte el lexer en `INDENT`/`DEDENT`. Dos predicados semánticos resuelven lo que la sangría no dice: `sameLine()`, para que `retornar` solo tome un valor de su misma línea, y `continues()`, para que un `+` o un `-` al inicio de una línea continúe la expresión anterior solo dentro de un paréntesis abierto.

```
parser grammar YParser;

options { tokenVocab = YLexer; }

/*
 * Y? delimita los bloques por sangria: YLexer convierte la sangria en INDENT /
 * DEDENT y oculta los saltos de linea. Por eso ninguna regla ve un fin de
 * linea: una instruccion termina donde ya no puede continuar, y el punto y
 * coma es opcional.
 *
 * Reglas ya decididas y que no hay que volver a discutir:
 *   - %estructuras es opcional, %funciones es obligatoria
 *   - las estructuras SOLO se declaran con 'estructura Nombre:' y campos
```

```
 *     indentados (aclaracion de la auxiliar, 5/09/2026); la forma tipo C se
 *     reconoce solo para reportarla con un mensaje propio
 *   - los arreglos se pasan como '[] entero arr' y las estructuras como
 *     '{} MiEstructura s', ambos por referencia
 *
 * Limites conocidos, que ningun ejemplo del enunciado toca: una '}' sangrada
 * al nivel de los 'caso', y una linea que empieza con '(' justo despues de una
 * expresion (se leeria como llamada). Tampoco hay ++ prefijo: pegado a la linea
 * anterior cambiaria su significado. Un literal { } si puede partirse en
 * varias lineas: YLexer no cuenta la sangria dentro de el.
 */

// (codigo Java de apoyo omitido: ver el archivo .g4)

/* ================================================================
 * 1. ESTRUCTURA DEL ARCHIVO
 * ================================================================ */
programa : seccionEstructuras? seccionFunciones EOF ;

seccionEstructuras : SECCION_ESTRUCTURAS declaracionEstructura* ;
seccionFunciones   : SECCION_FUNCIONES declaracionFuncion* ;

/* ================================================================
 * 2. ESTRUCTURAS
 * ================================================================ */
declaracionEstructura
    : ESTRUCTURA ID DOS_PUNTOS INDENT campoEstructura+ DEDENT               # estructuraIndentada
    | kw=ESTRUCTURA LLAVE_IZQ INDENT? campoEstructura* DEDENT? LLAVE_DER ID PUNTO_COMA?
      { notifyErrorListeners($kw, "Las estructuras en Y? se declaran con 'estructura Nombre:' y
          campos indentados.", null); }
                                                                             # estructuraEstiloC
    ;

// entero miArray[ 10 ]  -- dentro de una estructura el tamano es constante
campoEstructura : tipo ID ( COR_IZQ ENTERO COR_DER )? PUNTO_COMA? ;

tipo : ENTERO_T | CADENA_T | FLOTANTE_T | CARACTER_T | BOOL_T | ID ;

/* ================================================================
 * 3. FUNCIONES
 * ================================================================ */
declaracionFuncion
    : DEFINIR ID PAR_IZQ listaParametros? PAR_DER ( FLECHA tipo )? DOS_PUNTOS bloque
    ;

listaParametros : parametro ( COMA parametro )* ;

// [] entero miArray  -> arreglo por referencia;  {} Persona p -> estructura por referencia
parametro : ( COR_IZQ COR_DER | LLAVE_IZQ LLAVE_DER )? tipo ID ;

bloque : INDENT instruccion+ DEDENT ;

/* ================================================================
 * 4. INSTRUCCIONES
 * La declaracion va antes que la expresion suelta: "Persona p" tambien
 * podria leerse como dos expresiones.
 * ================================================================ */
instruccion
    : ( declaracionEstructura
      | declaracionVariable
      | asignacion
      | instruccionIncremento
      | instruccionSi
      | instruccionElegir
      | instruccionPara
      | instruccionMientras
      | instruccionHacer
      | instruccionRetorno
      | instruccionRomper
      | instruccionContinuar
      | instruccionImprimir
      | instruccionExpresion
      | bloqueInesperado
      ) PUNTO_COMA?
    ;
```

```
// Una linea sangrada que nada abre: se reporta una vez y el bloque se lee igual,
// en vez de dejar que ANTLR se tope con su INDENT y luego con su DEDENT.
bloqueInesperado
    : kw=INDENT instruccion+ DEDENT
      { notifyErrorListeners($kw, "Sangria inesperada: la linea anterior no abre ningun
                                                        bloque.", null); }
    ;

declaracionVariable
    : tipo ID ( COR_IZQ expresion COR_DER )+ ( ASIGNACION literalCompuesto )?   # declaracionArreglo
    | tipo ID ( ASIGNACION expresion )?                                         # declaracionSimple
    ;

asignacion : destino ASIGNACION expresion ;

destino : ID sufijoDestino* ;

sufijoDestino
    : COR_IZQ expresion COR_DER   # accesoIndice
    | PUNTO ID                    # accesoAtributo
    ;

instruccionIncremento : destino ( INCREMENTO | DECREMENTO ) ;

instruccionSi
    : SI PAR_IZQ expresion PAR_DER ENTONCES bloque
      sinoCondicional*
      ( CONTRARIO bloque )?
    ;

sinoCondicional : SINO PAR_IZQ expresion PAR_DER ENTONCES bloque ;

// Un caso sin cuerpo cae al siguiente, como en C.
// The cases go between braces, or after ':' and indented like any other Y? block.
instruccionElegir
    : ELEGIR PAR_IZQ expresion PAR_DER
      ( LLAVE_IZQ INDENT? casoElegir* casoSiempre? DEDENT? LLAVE_DER
      | DOS_PUNTOS INDENT casoElegir* casoSiempre? DEDENT
      )
    ;

casoElegir  : CASO expresion DOS_PUNTOS bloque? ;
casoSiempre : SIEMPRE DOS_PUNTOS bloque? ;

// Reglas propias para la cabecera: con 'instruccion' su ';' opcional se comeria el separador.
instruccionPara
    : PARA PAR_IZQ inicializacionPara PUNTO_COMA expresion PUNTO_COMA actualizacionPara PAR_DER
      DOS_PUNTOS bloque
    ;

inicializacionPara
    : tipo ID ASIGNACION expresion   # inicioDeclaracion
    | destino ASIGNACION expresion   # inicioAsignacion
    ;

actualizacionPara
    : destino ( INCREMENTO | DECREMENTO )   # actualizacionUnaria
    | destino ASIGNACION expresion          # actualizacionAsignacion
    ;

instruccionMientras : MIENTRAS PAR_IZQ expresion PAR_DER HACER bloque ;

instruccionHacer : HACER DOS_PUNTOS bloque MIENTRAS PAR_IZQ expresion PAR_DER ;

instruccionRetorno   : RETORNAR ( {sameLine()}? expresion )? ;
instruccionRomper    : ROMPER ;
instruccionContinuar : CONTINUAR ;
instruccionImprimir  : IMPRIMIR PAR_IZQ listaExpresiones? PAR_DER ;

// Cualquier expresion: el semantico decide cual puede ir sola.
instruccionExpresion : expresion ;

/* ================================================================
 * 5. EXPRESIONES  -- misma cadena de precedencia que PigLatin
 * ================================================================ */
expresion : expresionOr ;
```

```
expresionOr             : expresionAnd ( OR expresionAnd )* ;
expresionAnd            : expresionIgualdad ( AND expresionIgualdad )* ;
expresionIgualdad       : expresionRelacional ( ( IGUALDAD | DIFERENTE ) expresionRelacional )* ;
expresionRelacional     : expresionAditiva ( ( MENOR | MAYOR | MENOR_IGUAL | MAYOR_IGUAL )
                          expresionAditiva )* ;
// Sin fin de instruccion, "-b" en su propia linea se pegaria a la anterior: ver continues().
expresionAditiva        : expresionMultiplicativa ( {continues()}? ( MAS | MENOS )
                          expresionMultiplicativa )* ;
expresionMultiplicativa : expresionUnaria ( ( POR | DIVISION ) expresionUnaria )* ;

expresionUnaria
    : NEGACION expresionUnaria   # unariaNegacion
    | MENOS expresionUnaria      # unariaNegativo
    | expresionSufijo            # unariaSufijoDelegado
    ;

expresionSufijo : expresionPrimaria sufijoExpresion* ;

sufijoExpresion
    : COR_IZQ expresion COR_DER   # sufijoIndice
    | PUNTO ID                    # sufijoAtributo
    ;

expresionPrimaria
    : ENTERO                                  # primariaEntero
    | DECIMAL                                 # primariaDecimal
    | TEXTO                                   # primariaTexto
    | CARACTER                                # primariaCaracter
    | VERDADERO                               # primariaVerdadero
    | FALSO                                   # primariaFalso
    | LEER PAR_IZQ PAR_DER                    # primariaLeer
    | ID PAR_IZQ listaExpresiones? PAR_DER    # primariaLlamada
    | ID                                      # primariaIdentificador
    | PAR_IZQ expresion PAR_DER               # primariaAgrupacion
    | literalCompuesto                        # primariaLiteral
    ;

// Y? solo tiene la forma posicional: Punto p1 = { 10, 20, 85.5 }
literalCompuesto : LLAVE_IZQ listaExpresiones? LLAVE_DER ;

listaExpresiones : expresion ( COMA expresion )* ;
```

Zetariano (ZParser.g4)

```
parser grammar ZParser;

options { tokenVocab = ZLexer; }

/*
 * Zetariano: una clase por archivo, sintaxis de Java.
 *
 * Reglas ya decididas:
 *   - encapsulamiento: public, private, protected o ninguno (default) en
 *     cada atributo, metodo y constructor; el semantico valida el acceso
 *   - 'this' es el objeto del metodo: this.x, this.m() y this solo
 *   - herencia simple: class Perro extends Animal. super(...) llama al
 *     constructor del padre y super.m(...) a su version de un metodo; el
 *     semantico exige el @Override de un metodo que sobrescribe a otro
 *   - dangling else: IF ... instruccion ( ELSE instruccion )? es voraz, asi el
 *     'else' queda con el 'if' mas cercano, como en Java
 *   - 'switch' con fallthrough: 'break' es opcional
 *   - 'for ( ; ; )': las tres partes son opcionales
 *   - un arreglo se crea con '{...}' solo al declararlo; 'new T[n]' es una
 *     expresion mas: celdas = new int[f][c];
 */

/* ========================================================================
 * 1. CLASE
 * ======================================================================== */
programa : declaracionClase EOF ;

declaracionClase : PUBLIC? CLASS ID herencia? LLAVE_IZQ miembro* LLAVE_DER ;
```

```
herencia : EXTENDS ID ;

// La lectura adelantada distingue las tres: "Pila(" es constructor,
// "tipo nombre(" es metodo y "tipo nombre =" o "tipo nombre;" es atributo.
miembro
    : visibilidad? ID PAR_IZQ listaParametros? PAR_DER bloque  # constructor
    | OVERRIDE? visibilidad? ( tipo | VOID ) ID PAR_IZQ listaParametros? PAR_DER bloque # metodo
    | visibilidad? tipo ID ( ASIGNACION inicializador )? PUNTO_COMA                     # atributo
    ;

visibilidad : PUBLIC | PRIVATE | PROTECTED ;

// int[][] -- cada par de corchetes es una dimension
tipo     : tipoBase ( COR_IZQ COR_DER )* ;
tipoBase : INT_T | DOUBLE_T | CHAR_T | BOOLEAN_T | STRING_T | ID ;

listaParametros : parametro ( COMA parametro )* ;
parametro       : tipo ID ;

/* ======================================================================
 * 2. INSTRUCCIONES
 * ====================================================================== */
bloque : LLAVE_IZQ instruccion* LLAVE_DER ;

instruccion
    : bloque                                                                    # instruccionBloque
    | declaracionLocal PUNTO_COMA  # instruccionDeclaracion
    | asignacion PUNTO_COMA  # instruccionAsignacion
    | IF PAR_IZQ expresion PAR_DER instruccion ( ELSE instruccion )?            # instruccionSi
    | SWITCH PAR_IZQ expresion PAR_DER LLAVE_IZQ casoSwitch* LLAVE_DER          # instruccionSwitch
    | FOR PAR_IZQ inicioFor? PUNTO_COMA expresion? PUNTO_COMA actualizacionFor? PAR_DER
      instruccion                                                               # instruccionFor
    | WHILE PAR_IZQ expresion PAR_DER instruccion                               # instruccionWhile
    | DO instruccion WHILE PAR_IZQ expresion PAR_DER PUNTO_COMA                 # instruccionDoWhile
    | RETURN expresion? PUNTO_COMA                                              # instruccionRetorno
    | BREAK PUNTO_COMA                                                          # instruccionBreak
    | CONTINUE PUNTO_COMA                                                       # instruccionContinue
    | ( PRINT | PRINTLN ) PAR_IZQ expresion? PAR_DER PUNTO_COMA                 # instruccionImprimir
    | SUPER PAR_IZQ listaExpresiones? PAR_DER PUNTO_COMA                        # instruccionSuper
    | expresion PUNTO_COMA  # instruccionExpresion
    | PUNTO_COMA                                                                # instruccionVacia
    ;

declaracionLocal : tipo ID ( ASIGNACION inicializador )? ;

// Un arreglo nace al declararse: con su tamano o con sus valores.
inicializador
    : NEW tipoBase ( COR_IZQ expresion COR_DER )+   # inicializadorNuevoArreglo
    | literalArreglo                                # inicializadorLiteral
    | expresion                                     # inicializadorExpresion
    ;

literalArreglo  : LLAVE_IZQ ( elementoArreglo ( COMA elementoArreglo )* )? LLAVE_DER ;
elementoArreglo : literalArreglo | expresion ;

asignacion : destino op=( ASIGNACION | MAS_IGUAL | MENOS_IGUAL | POR_IGUAL ) expresion ;

destino : ( THIS PUNTO )? ID sufijoDestino* ;

sufijoDestino
    : COR_IZQ expresion COR_DER   # accesoIndice
    | PUNTO ID                    # accesoAtributo
    ;

casoSwitch
    : CASE expresion DOS_PUNTOS instruccion*   # caso
    | DEFAULT DOS_PUNTOS instruccion*          # casoDefault
    ;

inicioFor       : declaracionLocal | asignacion ;
actualizacionFor : asignacion | expresion ;

/* ======================================================================
 * 3. EXPRESIONES  -- precedencia de Java, de menor a mayor
 * ====================================================================== */
```

```
expresion : expresionTernaria ;

expresionTernaria : expresionOr ( INTERROGA expresion DOS_PUNTOS expresionTernaria )? ;

expresionOr             : expresionAnd ( OR expresionAnd )* ;
expresionAnd            : expresionIgualdad ( AND expresionIgualdad )* ;
expresionIgualdad       : expresionRelacional ( ( IGUALDAD | DIFERENTE ) expresionRelacional )* ;
expresionRelacional     : expresionAditiva ( ( MENOR | MAYOR | MENOR_IGUAL | MAYOR_IGUAL )
                          expresionAditiva )* ;
expresionAditiva        : expresionMultiplicativa ( ( MAS | MENOS ) expresionMultiplicativa )* ;
expresionMultiplicativa : expresionUnaria ( ( POR | DIVISION | MODULO ) expresionUnaria )* ;

expresionUnaria
    : NEGACION expresionUnaria                      # unariaNegacion
    | MENOS expresionUnaria                         # unariaNegativo
    | ( INCREMENTO | DECREMENTO ) expresionUnaria   # unariaPrefija
    | expresionSufijo                               # unariaSufijoDelegado
    ;

expresionSufijo : expresionPrimaria sufijoExpresion* ;

sufijoExpresion
    : COR_IZQ expresion COR_DER                   # sufijoIndice
    | PUNTO ID PAR_IZQ listaExpresiones? PAR_DER  # sufijoMetodo
    | PUNTO ID                                    # sufijoAtributo
    | INCREMENTO                                  # sufijoIncremento
    | DECREMENTO                                  # sufijoDecremento
    ;

expresionPrimaria
    : ENTERO                                        # primariaEntero
    | DECIMAL                                       # primariaDecimal
    | TEXTO                                         # primariaTexto
    | CARACTER                                      # primariaCaracter
    | TRUE                                          # primariaVerdadero
    | FALSE                                         # primariaFalso
    | NULL                                          # primariaNulo
    | THIS                                          # primariaThis
    | SUPER PUNTO ID PAR_IZQ listaExpresiones? PAR_DER # primariaSuper
    | READLN PAR_IZQ PAR_DER                        # primariaLeer
    | NEW tipoBase ( COR_IZQ expresion COR_DER )+ # primariaNuevoArreglo
    | NEW ID PAR_IZQ listaExpresiones? PAR_DER    # primariaNuevo
    | ID PAR_IZQ listaExpresiones? PAR_DER        # primariaLlamada
    | ID                                          # primariaIdentificador
    | PAR_IZQ expresion PAR_DER                   # primariaAgrupacion
    ;

listaExpresiones : expresion ( COMA expresion )* ;
```

Tabla de compatibilidad de tipos

Los tres lenguajes comparten un único sistema de tipos (`DataType` y `TypeSystem`): cada tipo de un lenguaje es el mismo tipo interno con otro nombre. Las tablas se generaron ejecutando `TypeSystem` con cada combinación, así que son las reglas que aplica el compilador. Una celda con «error» es un error semántico.

Los tipos de cada lenguaje y su representación en RISC-V.

Tipo interno	PigLatin	Y?	Zetariano	En RISC-V
`NUMERUS`	`numerus`	`entero`	`int`	entero de 32 bits
`DECIMALIS`	`decimalis`	`flotante`	`double`	float IEEE-754 de 32 bits
`LITTERA`	`littera`	`caracter`	`char`	código del carácter
`TEXTUM`	`textum`	`cadena`	`String`	dirección en el heap; un carácter por celda y un 0 al final
`BOOLEANO`	`bool` (y los antiguos `verum`/`falsus`)	`bool`	`boolean`	1 o 0
`ESTRUCTURA`	estructura de Y? u objeto	estructura	objeto	dirección del bloque en el heap
`NULO`	`null`	—	`null`	0, la celda nula

La jerarquía de conversión implícita es `caracter → entero → decimal`: el resultado de una operación numérica es el operando de mayor rango. Una división entre dos enteros es entera (`17 / 2` da `8`). Dos caracteres que se suman dan un entero, como en Java.

Tablas por lenguaje

PigLatin

PigLatin: resultado de a + b (filas: a, columnas: b).

a \ b	numerus	decimalis	littera	textum	bool
numerus	numerus	decimalis	numerus	textum	error
decimalis	decimalis	decimalis	decimalis	textum	error
littera	numerus	decimalis	numerus	textum	error
textum	textum	textum	textum	textum	textum
bool	error	error	error	textum	error

PigLatin: resultado de a - b, a * b y a / b.

a \ b	numerus	decimalis	littera	textum	bool
numerus	numerus	decimalis	numerus	error	error

a \ b	numerus	decimalis	littera	textum	bool
decimalis	decimalis	decimalis	decimalis	error	error
littera	numerus	decimalis	numerus	error	error
textum	error	error	error	error	error
bool	error	error	error	error	error

PigLatin: comparaciones y operadores lógicos.

Operación	Operandos válidos	Resultado
`== !=`	dos numéricos (numerus, decimalis, littera, mezclados), dos textum, dos bool, dos objetos o un objeto y null	bool
`< > <= >=`	dos numéricos (numerus, decimalis, littera, mezclados)	bool
`&& || non`	bool con bool	bool
- unario	numerus, decimalis; littera da numerus	el mismo tipo
`++ --`	numerus y decimalis	el mismo tipo

PigLatin: ¿se puede guardar un valor (columna) en un destino (fila)?

destino \ valor	numerus	decimalis	littera	textum	bool
numerus	sí	no	sí	no	no
decimalis	sí	sí	sí	no	no
littera	no	no	sí	no	no
textum	no	no	no	sí	no
bool	no	no	no	no	sí

- `textum + cualquier primitivo` concatena: el número, el carácter o el valor lógico se convierte a texto (`"Edad: " + 33`, `"listo: " + verum`).
- `null` solo se asigna a un objeto de una clase de Zetariano: ni a un primitivo, `textum` incluido, ni a una estructura de Y?. Vale en la declaración (`esto x : null : Persona;`), en una asignación, como argumento y en `si (x == null)`.
- `esto x : novus Profesor(..) : Persona;` declara `x` como `Persona`: el `novus` tiene que crear esa clase o una hija suya.

Y?

Y?: resultado de a + b (filas: a, columnas: b).

a \ b	entero	flotante	caracter	cadena	bool
entero	entero	flotante	entero	cadena	error
flotante	flotante	flotante	flotante	cadena	error

a \ b	entero	flotante	caracter	cadena	bool
caracter	entero	flotante	entero	cadena	error
cadena	cadena	cadena	cadena	cadena	cadena
bool	error	error	error	cadena	error

Y?: resultado de a - b, a * b y a / b.

a \ b	entero	flotante	caracter	cadena	bool
entero	entero	flotante	entero	error	error
flotante	flotante	flotante	flotante	error	error
caracter	entero	flotante	entero	error	error
cadena	error	error	error	error	error
bool	error	error	error	error	error

Y?: comparaciones y operadores lógicos.

Operación	Operandos válidos	Resultado
`== !=`	dos numéricos (entero, flotante, caracter, mezclados), dos cadena, dos bool, dos estructuras	bool
`< > <= >=`	dos numéricos (entero, flotante, caracter, mezclados)	bool
`&& || !`	bool con bool	bool
- unario	entero, flotante; caracter da entero	el mismo tipo
`++ --`	entero y flotante	el mismo tipo

Y?: ¿se puede guardar un valor (columna) en un destino (fila)?

destino \ valor	entero	flotante	caracter	cadena	bool
entero	sí	no	sí	no	no
flotante	sí	sí	sí	no	no
caracter	no	no	sí	no	no
cadena	no	no	no	sí	no
bool	no	no	no	no	sí

- `cadena + cualquier primitivo` concatena.
- Una estructura solo se asigna a una variable de la misma estructura.
- Los arreglos y estructuras que llegan como parámetro (`[] entero arr`, `{} Punto p`) se pasan por referencia.

Zetariano

Zetariano: resultado de a + b (filas: a, columnas: b).

a \ b	int	double	char	String	boolean
int	int	double	int	String	error
double	double	double	double	String	error
char	int	double	int	String	error
String	String	String	String	String	String
boolean	error	error	error	String	error

Zetariano: resultado de a - b, a * b y a / b.

a \ b	int	double	char	String	boolean
int	int	double	int	error	error
double	double	double	double	error	error
char	int	double	int	error	error
String	error	error	error	error	error
boolean	error	error	error	error	error

Zetariano: resultado de a % b (residuo entero).

a \ b	int	double	char	String	boolean
int	int	error	int	error	error
double	error	error	error	error	error
char	int	error	int	error	error
String	error	error	error	error	error
boolean	error	error	error	error	error

Zetariano: comparaciones y operadores lógicos.

Operación	Operandos válidos	Resultado
`== !=`	dos numéricos (int, double, char, mezclados), dos String, dos boolean, dos objetos o un objeto y null	boolean
`< > <= >=`	dos numéricos (int, double, char, mezclados)	boolean
`&& || !`	boolean con boolean	boolean
- unario	int, double; char da int	el mismo tipo
`++ --`	int y double	el mismo tipo
`c ? a : b`	c boolean; a y b compatibles	el más ancho de los dos

Zetariano: ¿se puede guardar un valor (columna) en un destino (fila)?

destino \ valor	int	double	char	String	boolean
int	sí	no	sí	no	no
double	sí	sí	sí	no	no
char	no	no	sí	no	no
String	no	no	no	sí	no
boolean	no	no	no	no	sí

- `String + cualquier primitivo` concatena.
- `null` se asigna a un objeto o a un `String`.
- Un objeto de una subclase cabe donde se espera su clase padre (`Animal a = new Perro();`), nunca al revés.

Reglas que no dependen solo del tipo

- **Objetos.** Un objeto de una subclase cabe en una variable, un parámetro o un retorno de su clase padre o de una de sus ancestras; al revés es error: `'Animal' no es 'Perro' ni una de sus clases hijas`.
- **Arreglos completos.** Se comparten, no se copian, así que exigen el mismo tipo de elemento y el mismo número de dimensiones: un arreglo de enteros no cabe donde se espera uno de decimales. Un arreglo de objetos no admite subclases (`Perro[]` no cabe en `Animal[]`).
- **Condiciones.** Las de `si`, ciclos y ternario deben ser booleanas.
- **Lectura.** `leer()` o `readln()` toma el tipo de su destino; sola, devuelve texto.
- **Retorno y argumentos.** Siguen las mismas reglas que la asignación, con el tipo del retorno o del parámetro como destino.
- **Rangos.** Un entero literal va hasta 2147483647 (y `-2147483648`); un decimal literal tiene que caber en un float de 32 bits.

Análisis semántico

Dos pasadas

`CompilerPipeline` crea un `SemanticAnalyzer` por archivo, todos sobre la misma `SymbolTable`. La primera pasada (`register`) registra las estructuras, las clases con sus atributos y su padre, y las firmas de funciones, métodos y constructores de todos los archivos. La segunda (`visitProgram`) revisa los cuerpos. Por eso `Pila.z` puede usar a `Nodo` sin importarlo, y el orden de los `import` del `.pig` da igual.

Ámbitos y tabla de símbolos

`SymbolTable` mantiene una pila de ámbitos (`Scope`): el global, uno por función o método, uno por bloque y uno por clase. Las funciones tienen su propio espacio de nombres para que dos sobrecargas no choquen: la sobrecarga se distingue por los tipos de los parámetros, como en Java. Los atributos heredados entran al ámbito de la clase hija con `declareInherited`, sin repetirse en la tabla que muestra la interfaz.

Encapsulamiento

No hay paquetes (aclaración de la auxiliar del 15 de octubre), y un miembro sin modificador, default, funciona como `private`: solo se usa dentro de su clase (aclaraciones del 22 y 24 de octubre). `SemanticAnalyzer.checkAccess` es el único punto de control y se llama al acceder a un atributo, al llamar a un método y al crear un objeto; `Visibility.isClassOnly()` reúne la regla de `private` y default. Por eso el banco de pruebas de la auxiliar da dos errores: `CuentaCorriente` usa `movimientos`, que en `Cuenta` no tiene modificador. Declarado `protected`, compila y corre (`examples/p2/banco`).

Acceso a un miembro según su modificador.

Modificador	Misma clase	Clase hija	Otra clase	Desde el .pig
`public`	sí	sí	sí	sí
default (sin modificador)	sí	no	no	no
`protected`	sí	sí	no	no
`private`	sí	no	no	no

Herencia, @Override y super

- Herencia simple: `class Perro extends Animal`. El padre tiene que existir y no puede haber ciclos. Se reporta en cada clase del ciclo, y las búsquedas que suben por los padres se detienen donde el ciclo se cierra.
- La hija hereda los atributos y métodos `public` y `protected`. Los `private` y los default del padre siguen dentro del objeto, porque los usan los métodos del padre, pero la hija no puede nombrarlos ni sobrescribirlos: un método igual en la hija es otro método.
- `@Override` es obligatorio. Es error si falta, si sobra (no hay método que sobrescribir), si cambia el tipo de retorno o si reduce la visibilidad (`public` > `protected` > default = `private`). Si el padre retorna un objeto, la hija puede retornar una de sus clases hijas: el retorno covariante de Java (aclaración del 19 de octubre).
- Un constructor de la hija empieza llamando al del padre: con `super(..)` si lo escribe como primera instrucción, o con el `super()` implícito. Si el padre no tiene constructor sin parámetros, `super(..)` es obligatorio. Después corren los valores iniciales de los atributos de la hija y luego el resto del cuerpo.
- `super.m()` llama a la versión del padre sin despacho. `obj.m()`, `this.m()` y `m()` se despachan por la clase del objeto.

Manejo de errores

`ErrorManager` reúne todos los errores con su tipo, el archivo, la línea y la columna (desde 1), el lexema y la descripción. La pestaña "Errores" los muestra en ese orden. Así se evita que un error genere otros:

- Un error léxico corta el archivo: ese archivo no se parsea, y si viene de un import no se analiza nada más.
- Un error sintáctico no corta: ANTLR se recupera, el AST se construye con lo reunido y el semántico reporta lo demás.

- Una expresión con un error toma el tipo `ERROR`, que es compatible con todo, para que un solo error no produzca diez mensajes.
- Un fallo interno del compilador se muestra como un error del archivo en vez de cerrar la aplicación, y queda en el log con su traza completa.

Pila de procesos

ANTLR 4 es LL(*) y no tiene una pila LR que mostrar. `ProcessStackListener` la simula recorriendo el árbol ya construido, para que una predicción que retrocede no duplique pasos. Cada paso guarda una copia de la pila, así que avanzar o retroceder es solo mover un índice.

De evento del recorrido a acción de la pila.

Evento de ANTLR	**Acción**	**Descripción**
`enterEveryRule`	PUSH	Se activa la regla: entra a la pila.
`visitTerminal`	SHIFT	Se consume un token.
`exitEveryRule`	REDUCE	La regla se reduce a un nodo del árbol y sale de la pila.
fin de `programa` sin errores	ACCEPT	La cadena es aceptada por la gramática.
`visitErrorNode`	ERROR	Token inesperado.

Decisiones y desviaciones

Decisiones tomadas donde el enunciado deja lugar a interpretación.

Tema	**Decisión**
Visibilidad default	Funciona como `private`: solo dentro de su clase, sin herencia ni sobrescritura (aclaraciones del 22 y 24 de octubre). No hay paquetes (15 de octubre).
Declaraciones en `MAIOR>`	Se permiten en cualquier parte del bloque principal. Dentro de una función van al inicio, en `VARIABILES[ ]`.
Argumentos	A lo sumo 8, contando el `this` de un método: RISC-V los pasa en `a0`-`a7`. Una función que necesita más es error semántico.
`decimalis`	Float de 32 bits, como la celda donde se guarda.
Lectura de enteros	Como `strtol` de C: salta blancos, toma signo y dígitos, y da 0 si no hay número. Cada lectura consume una línea completa.
`null` en PigLatin	Solo para objetos de una clase.
Retorno al sobrescribir	El mismo tipo o, si es un objeto, una de sus clases hijas: retorno covariante (aclaración del 19 de octubre).
Métodos private y default	No se heredan: uno igual en la hija es otro método, sin `@Override` y sin despacho.

Tema	Decisión
Estructuras en Y?	Solo `estructura Nombre:` con los campos sangrados; la forma tipo C se reconoce para reportarla con un mensaje propio.
`elegir` en Y?	Acepta `elegir(x) { ... }` y `elegir(x) :` con los casos sangrados.

Código de tres direcciones y cuartetas

El código intermedio se construye "a mano", siguiendo las reglas clásicas del código de tres direcciones. Cada instrucción es una cuarteta `(op, arg1, arg2, resultado)`: la pestaña "Cuartetas" y el listado de "Codigo 3D" son la misma lista vista de dos formas.

- Una operación por instrucción: `x = a + b * c` se escribe `t1 = b * c`, `t2 = a + t1`, `x = t2`.
- Las variables van por su nombre. Los temporales `t1, t2...` y las etiquetas `L1, L2...` tienen un contador único en todo el programa.
- Las condiciones son saltos: se salta al caso falso con la condición invertida, y `&&` y `||` van en corto circuito.
- Todo lo que hace el programa está en el C3D: la memoria, las cadenas y las conversiones. Solo imprimir o leer un número o un carácter es una instrucción primitiva.

Instrucciones del código de tres direcciones.

Instrucción	**op**	**Texto C3D**
Aritmética entera	`+ - * / %`	`x = a + b`
Aritmética flotante	`+f -f *f /f`	`x = a +f b`
Menos unario y conversión	`uminus uminusf itof ftoi`	`x = itof a`
Copia	`=`	`x = a`
Lectura y escritura del heap	`=[]` y `[]=`	`x = heap[i]` / `heap[i] = v`
Etiqueta y salto	`label goto`	`L1:` / `goto L1`
Salto condicional	`if< if<= if> if>= if== if!=` (y `iff..` flotantes)	`if a < b goto L1`
Global	`global`	`global x`
Función	`func end_func`	`func f(p1, p2):` ... `end_func f`
Llamada	`param call return`	`param a`, `x = call f, 1`, `return a`
E/S	`print_int print_char read_int read_float read_char read_str`	`print_int a`, `x = read_int`
Fin	`halt`	`halt`

Modelo de memoria

Dónde vive cada cosa.

Qué	Dónde	En el C3D
Parámetros, variables locales y temporales	Registro de activación de la función, en la pila RISC-V (relativo a `fp`)	por su nombre: `n`, `t3`
Globales de `VARIABILES>`	Segmento `.data`	`global edad` y luego `edad`
Cadenas, arreglos, estructuras y objetos	Heap: celdas de 4 bytes, sin liberación	`heap[i]`; reservar `n` celdas es `p = H` y `H = H + n`

- `heap[0] = 0` es `null`, y leído como cadena es `""`.
- Cadena: un carácter por celda y un 0 al final. Las constantes viven en un pool al inicio del heap (celdas `1..k`).
- Arreglo de rango k: `[d0, d1, …, dk-1 | e0, e1, …]`. La variable apunta a `e0`; la dimensión `j` está en `heap[p - k + j]`. `m[i][j]` vive en `m + i*d1 + j`, con `d1` leído en tiempo de ejecución, por eso las dimensiones pueden ser variables.
- Estructura: `[campo0, campo1, …]`. Objeto: `[idClase | campos del padre | campos propios]`. Un campo heredado queda en el mismo desplazamiento que en el padre, y `idClase` sirve para el despacho polimórfico.

Condiciones y ciclos

`saltoFalso(E, L)` salta a `L` si `E` es falsa y, si no, sigue de largo; `saltoVerdadero` es su opuesto. `E1 && E2` en falso es `saltoFalso(E1, L); saltoFalso(E2, L)`. `interrumpe`/`romper`/`break` saltan a la salida del ciclo o switch más cercano, y `perge`/`continuar`/`continue` a su punto de continuación. El `switch` tiene fallthrough: un caso sin `break` cae al siguiente.

```
dum (i < 10) { C } finis;

L1:
    if i >= 10 goto L2
    <C>
    goto L1
L2:             <- interrumpe
```

```
per (esto i : numerus 0; i < 10; i++) { C }

    i = 0
L1:
    if i >= 10 goto L3
    <C>
L2:                 <- perge
    t1 = i + 1
    i = t1
    goto L1
L3:                 <- interrumpe
```

Funciones y llamadas

Los argumentos se evalúan todos primero y solo entonces se emiten los `param`, juntos y en orden, justo antes del `call`, para que una llamada anidada no intercale los suyos. Un método se nombra `Clase_metodo` y recibe su objeto como primer parámetro, `this`. Si una función tiene sobrecargas, su nombre lleva los tipos de sus parámetros (`insertarFinal__int`).

```
definir factorial(entero n) -> entero:
    si(n < 2) entonces
        retornar 1
    retornar n * factorial(n - 1)
```

```
func factorial(n):
    if n >= 2 goto L1
    return 1
L1:
    t1 = n - 1
    param t1
    t2 = call factorial, 1
    t3 = n * t2
    return t3
end_func factorial
```

Objetos y despacho polimórfico

`novus`/`new` reserva el bloque, escribe el `idClase` en su primera celda y llama al constructor con el bloque como `this`. Si alguna subclase del tipo estático sobrescribe el método, la llamada compara el `idClase` del objeto y salta a la versión de su clase. Las clases que comparten una implementación van a la misma etiqueta, y si ninguna sobrescribe el método la llamada es directa. Las comparaciones no tocan `a0`-`a7`, así que cada `call` encuentra los `param` ya cargados.

```
t13 = heap[t12]                 <- idClase del objeto
param t12
if t13 == 2 goto L5             <- ImpresoraPDF
if t13 == 4 goto L5             <- ImpresoraPortatil, que hereda la de ImpresoraPDF
if t13 == 3 goto L6             <- ImpresoraExcel
call Impresora_imprimir, 1
goto L4
L5: call ImpresoraPDF_imprimir, 1
goto L4
L6: call ImpresoraExcel_imprimir, 1
L4:
```

Funciones auxiliares

Se generan en C3D, como cualquier otra función, solo si el programa las usa. Ninguna está escrita en otro lenguaje.

Funciones auxiliares del C3D.

Función	Hace
`__print_str(s)`	Recorre la cadena con `print_char`.
`__concat(a, b)`	Copia `a` y `b` a partir de `H` y mueve `H`.
`__int_to_str(n)`	Escribe los dígitos de derecha a izquierda y el signo.
`__float_to_str(f)`	Signo, parte entera, punto y hasta 6 decimales redondeados, sin ceros de más.
`__char_to_str(c)`	Cadena de un carácter.
`__str_eq(a, b)`	1 si son iguales carácter por carácter.

Ejemplo completo

```
VARIABILES>
esto i : numerus 0;
esto total : numerus 0;
MAIOR>
dum (i < 10) {
    si (i > 5) { total = total + i; } finis;
    i = i + 1;
} finis;
>> "Total: " >> total ;
FINIS;
```

Sus cuartetas.

#	op	arg1	arg2	resultado	C3D
0	global			i	`global i`
1	global			total	`global total`
2	func			main	`func main():`
3	=	0		i	`i = 0`
4	=	0		total	`total = 0`
5	label			L1	`L1:`
6	if>=	i	10	L2	`if i >= 10 goto L2`
7	if<=	i	5	L3	`if i <= 5 goto L3`
8	+	total	i	t1	`t1 = total + i`
9	=	t1		total	`total = t1`
10	label			L3	`L3:`
11	+	i	1	t2	`t2 = i + 1`
12	=	t2		i	`i = t2`
13	goto			L1	`goto L1`
14	label			L2	`L2:`
15	param	"Total: "			`param "Total: "`
16	call	__print_str	1		`call __print_str, 1`
17	print_int	total			`print_int total`

#	op	arg1	arg2	resultado	C3D
18	print_char	10			`print_char 10`
19	halt				`halt`
20	end_func			main	`end_func main`

Detrás de `main` va `__print_str`, que es la única auxiliar que este programa usa. Las cuartetas 6 y 7 en RISC-V (`i` y `total` son globales):

```
L1:
    la   t6, g_i
    lw   t0, 0(t6)
    li   t1, 10
    blt  t0, t1, _s1        # if i >= 10 goto L2, invertida para usar j
    j    L2
_s1:
    la   t6, g_i
    lw   t0, 0(t6)
    li   t1, 5
    bgt  t0, t1, _s2
    j    L3
_s2:
```



La pestaña "Codigo 3D" con el listado de p2/integrador.

Traducción a RISC-V

`RiscVEmitter` traduce cada cuarteta a unas pocas instrucciones: carga los operandos en registros temporales, opera y guarda el resultado. Encima de cada traducción deja un comentario con su línea de C3D, para leer el ensamblador junto al código del que viene.

Uso de los registros.

Registro	Uso
`sp`, `fp` (`s0`), `ra`	Pila, registro de activación y dirección de retorno.
`s11`	Dirección base del heap, que devuelve `sbrk` al arrancar.
`t0`-`t6`, `ft0`-`ft2`	Traducción de cada cuarteta; no guardan nada entre una cuarteta y otra.
`a0`-`a7`, `fa0`	Argumentos (`a0`-`a7`), valor de retorno (`a0`) y llamadas al sistema.

Registro de activación

Cada `param` carga su valor en el siguiente registro `a0`-`a7`, y el prólogo de la función los copia a su marco. Todo `return` salta a la única salida de la función, donde está el epílogo. Si un desplazamiento no cabe en los 12 bits de `lw`/`sw`, se usa `li t6, off`, `add t6, t6, fp`.

```
fp - 4        ra guardado
fp - 8        fp guardado
fp - 12       param 1      <- copiado de a0 por el prologo
   ...
              param n      <- copiado de a(n-1)
              ranuras      <- cada local y cada temporal tiene una
sp

fn_f:                          # prologo
    sw   ra, -4(sp)
    sw   fp, -8(sp)
    mv   fp, sp
    li   t6, MARCO             # 8 + 4 * ranuras
    sub  sp, sp, t6
    sw   a0, -12(fp)           # cada parametro, a su ranura
    ...
_exit_fn_f:                    # epilogo, uno solo
    lw   ra, -4(fp)
    lw   t0, -8(fp)
    mv   sp, fp
    mv   fp, t0
    ret
```

De cuarteta a instrucciones

Traducción de cada cuarteta.

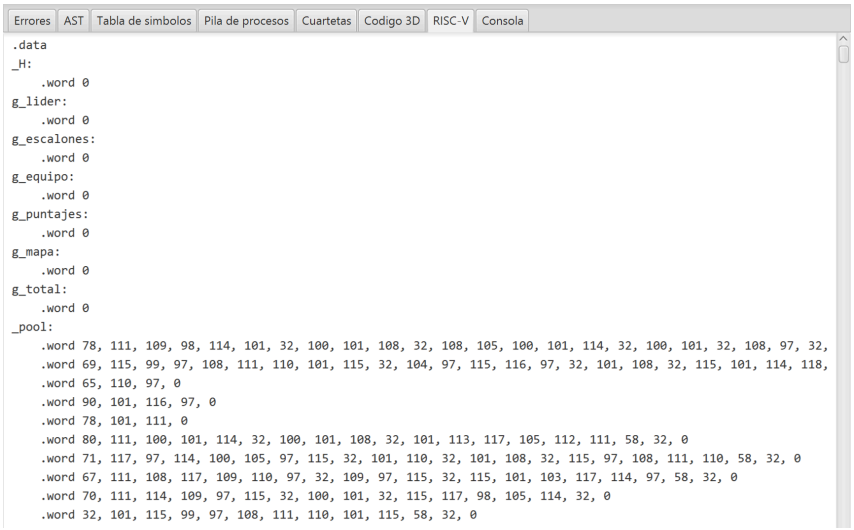
C3D	RISC-V
`x = a + b` (`- * / %`)	cargar a→t0 y b→t1; `add`/`sub`/`mul`/`div`/`rem t2, t0, t1`; guardar t2→x
`x = a +f b`	`fmv.w.x` a `ft0`/`ft1`; `fadd.s ft2, ft0, ft1`; `fmv.x.w t2, ft2`
`x = itof a` / `x = ftoi a`	`fcvt.s.w` / `fcvt.w.s ... rtz` (trunca hacia cero)
`x = heap[i]`	`slli t0, t0, 2`; `add t0, t0, s11`; `lw t1, 0(t0)`
`if a < b goto L`	`bge t0, t1, _sN`; `j L`; `_sN:`: la rama invertida salta un `j`, que alcanza ±1 MiB, porque una rama condicional solo alcanza ±4 KiB
`param a` / `x = call f, n`	cargar a en el siguiente `a0`-`a7` / `jal ra, fn_f` y guardar `a0`
`return a` / `end_func f`	cargar a→`a0`; `j _exit_fn_f` / la etiqueta de salida y el epílogo
`print_int a` / `print_char a`	cargar a→`a0`; `li a7, 1` / `11`; `ecall`
`x = read_int` / `read_str` / `read_char`	`jal ra, _rt_read_int` / `_rt_read_str` / `_rt_read_char`
`x = read_float`	`li a7, 6`; `ecall`; `fmv.x.w t0, fa0`
`halt`	`li a7, 10`; `ecall`

Arranque, lecturas y llamadas al sistema

El programa empieza en `main`: pide 3 MiB de heap con `sbrk` (786 432 celdas), copia el pool de cadenas a `heap[1..k]`, fija `H`, llama a `fn_main` y termina con `exit`. Las tres rutinas de lectura empiezan igual, leyendo una línea completa con `ReadString` en `_buffer`, y son las únicas escritas en ensamblador porque la llamada al sistema trabaja con bytes. `_rt_read_int` no usa `ReadInt` porque este detiene el programa ante una letra.

Llamadas al sistema de RARS que usa el ensamblador.

a7	Nombre	Entrada	Salida
1	PrintInt	`a0`	—
6	ReadFloat	—	`fa0`
8	ReadString	`a0` = buffer, `a1` = máximo	la línea en el buffer
9	Sbrk	`a0` = bytes	`a0` = dirección
10	Exit	—	—
11	PrintChar	`a0` (byte bajo)	—



La pestaña "RISC-V": cada bloque lleva encima, como comentario, la cuarteta de la que viene.

Ejecución del ensamblador

El ingeniero pidió que la aplicación compile y ejecute su ensamblador sin terminal ni programas externos, y que además haya un simulador listo para comprobar por fuera que el `.asm` es real. St4ck Over-Pig hace las dos cosas con RARS 1.6.

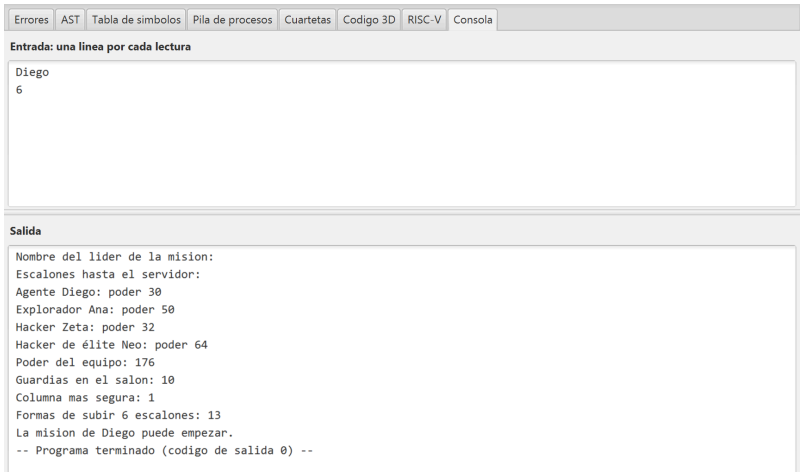
Dentro de la aplicación: RiscVRunner

El botón "Ejecutar" compila el archivo activo y, si se tradujo a RISC-V, entrega el ensamblador y el texto de "Entrada" de la pestaña "Consola" a `RiscVRunner.run(assembly, input)`. Este usa la API pública de RARS (`rars.api.Program`) dentro del mismo proceso:

```
Options options = new Options();
options.startAtMain = true;          // como la opcion "sm" de la consola
options.maxSteps    = MAX_STEPS;     // 11 000 000 instrucciones

Program program = new Program(options);
program.assembleString(assembly);    // AssemblyException si no ensambla
program.setup(new ArrayList<>(), input);
Simulator.Reason reason = program.simulate();   // SimulationException si falla al correr
program.getSTDOUT();                 // lo que imprimio, tambien si fallo a medias
```

- La entrada se da por adelantado, una línea por cada lectura, como el `.entrada.txt` de un ejemplo o `< archivo` en la consola.
- El resultado es un `RunResult(output, status, successful)`. La consola muestra la salida y una última línea `-- estado --`, al estilo de RARS, y la barra de estado la pinta de verde o de rojo.
- `MAX_STEPS` es unas 10 veces el ejemplo más pesado (`Laberinto`, 1 084 398 instrucciones). Un ciclo sin fin, o una lectura sin su línea, se detiene ahí con su mensaje en vez de correr para siempre.
- RARS guarda la máquina en campos estáticos, así que corre un programa a la vez. La ejecución va en un `javafx.concurrent.Task`, en un hilo aparte, para que la ventana siga respondiendo, y el botón "Ejecutar" se desactiva mientras tanto. El hilo es daemon: cerrar la ventana no espera a un programa que no termina.
- Medido en la máquina de desarrollo: el ejemplo más pesado tarda medio segundo, y llegar al tope de 11 millones de instrucciones, entre 4 y 13 segundos.



Ejecución interna de p2/integrador: su entrada arriba y lo que imprimió abajo.

Cómo termina una ejecución.

Caso	Mensaje	Color
El programa llega a su `exit`	`Programa terminado (codigo de salida 0)`	verde
Se alcanza `MAX_STEPS`	`Se detuvo tras 11 millones de instrucciones: un ciclo que no termina, o una lectura sin su linea en la entrada`	rojo
RARS lanza un error al correr	`Error de ejecucion: <mensaje de RARS>`, por ejemplo `invalid float input (syscall 6)`	rojo
RARS no ensambla el código	`RARS no pudo ensamblar el RISC-V generado`, con el reporte de RARS en la salida	rojo

Fuera de la aplicación: RARS

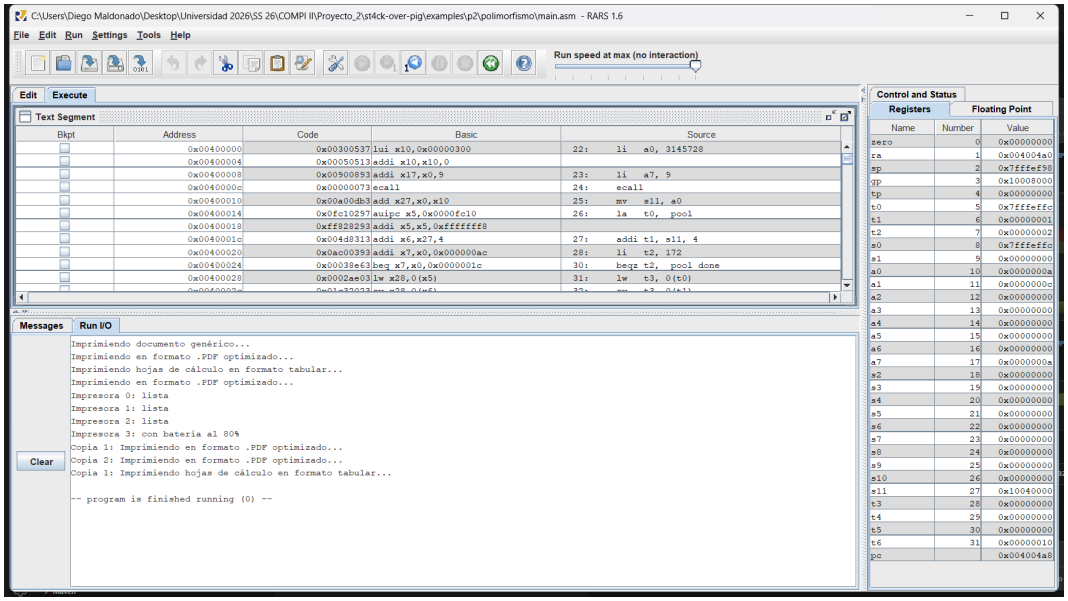
"Descargar .asm" guarda el RISC-V junto al archivo compilado, con su mismo nombre (`main.pig` → `main.asm`). El mismo jar de `lib/` es RARS completo:

- Interfaz: `java -jar lib/rars/rars/1.6/rars-1.6.jar`; luego *File* → *Open*, *Run* → *Assemble* (F3) y *Run* → *Go* (F5). Las lecturas se escriben en *Run I/O*.
- Consola (`cmd`, Git Bash, Linux o macOS): `java -Dstdout.encoding=UTF-8 -jar lib/rars/rars/1.6/rars-1.6.jar nc me sm se1 ae2 main.asm < main.entrada.txt`. Desde PowerShell, el mismo comando dentro de `cmd /c "..."`.

Opciones de RARS por consola.

Opción	Significado
`nc`	No imprime el aviso de copyright.

Opción	Significado
`me`	Manda los mensajes de RARS a la salida de error, separados de la salida del programa.
`sm`	Empieza en la etiqueta global `main`.
`se1` / `ae2`	Termina con código 1 ante un error de ejecución y con 2 ante uno de ensamblado.
`-Dstdout.encoding=UTF-8`	Opción de Java: los acentos salen en UTF-8 por consola.



El mismo .asm de p2/polimorfismo, ensamblado y ejecutado en la interfaz de RARS.

Por qué RARS y no Venus

El enunciado del P2 pide RISC-V "funcional", y el ingeniero mencionó Venus "o algún otro simulador". El ensamblador sigue la convención de llamadas al sistema de RARS: el número del servicio va en `a7` y el argumento en `a0`. Venus usa otra: el número va en `a0` y el argumento en `a1`, y no tiene lectura de consola. Por eso el `.asm` no corre en Venus sin cambiar cada `ecall`, y los programas que leen datos no correrían ahí. RARS es el simulador de referencia del curso, el mismo que corre dentro de la aplicación.

Pruebas

El proyecto no tiene una suite de JUnit. Cada cambio se prueba con programas temporales y con la regresión de `examples/`. Cada programa ejecutable tiene junto a él lo que se escribe en la consola (`<programa>.entrada.txt`, solo si lee algo) y lo que debe imprimir (`<programa>.salida.txt`). Los `errores.pig` no compilan a propósito: su cabecera lista cada error esperado, con su archivo, línea y columna.

Comprobaciones de la regresión final.

Comprobación	Resultado
C3D, `.asm` y errores de todos los ejemplos, comparados byte a byte con los de antes de cada cambio	idénticos en todo programa cuyo fuente no cambió
Los 18 programas en RARS por consola, contra su `.salida.txt`	18/18
Los 18 programas con la ejecución interna (`RiscVRunner`), contra su `.salida.txt`	18/18
Cada `errores.pig`, contra los errores de su cabecera	27, 7, 4 y 5 errores, sin cascadas
El banco de pruebas de la auxiliar (`03_banco`), tal como se entregó	2 errores: `movimientos` es default y `CuentaCorriente` lo usa (líneas 18 y 36)
Las aclaraciones de Telegram en programas temporales: default, `@Override`, retorno covariante y `double` con `int`	correcto
La ventana manejada por un script: "Ejecutar" sobre un `.pig` válido, uno con errores, un `.z`, un ciclo infinito, una salida de 650 000 caracteres y el banco de la auxiliar	correcto

Cómo correr la regresión

1. Abrir la carpeta `examples/` en la aplicación.
2. Para cada programa con `.salida.txt`: abrirlo, copiar su `.entrada.txt` (si existe) en "Entrada" de la pestaña "Consola" y pulsar "Ejecutar".
3. Comparar la salida con su `.salida.txt`. La última línea `-- Programa terminado (codigo de salida 0) --` no es parte del programa.
4. Por consola: "Descargar .asm" y `java -Dstdout.encoding=UTF-8 -jar lib/rars/rars/1.6/rars-1.6.jar nc me sm se1 ae2 main.asm < main.entrada.txt`, comparando con el `.salida.txt`.

Los ejemplos y lo que demuestra cada uno.

Ejemplo	Lenguajes	Qué demuestra
`enunciado.pig` y `lib/Funciones.y`	PigLatin, Y?	Los fragmentos del enunciado de los dos lenguajes: declaraciones, arreglos, estructuras, funciones y ciclos. Lee de la consola.
`estructuras_y_funciones.y`	Y?	Las estructuras y funciones del enunciado de Y?, analizadas solas: los bloques por sangría.
`resistencia.pig` y `resistencia.y`	PigLatin, Y?	Funciones, recursividad, arreglos, estructuras, los tres ciclos, `interrumpe`/`perge` y `elegir`. Lee un número.
`matrices/`	PigLatin, Y?	Matrices con sus valores fila por fila, aplanadas en el heap.
`objetos/`	PigLatin, Zetariano	Los fragmentos del enunciado de Zetariano: expresiones, arreglos, matrices, objetos, `null`, `switch` con fallthrough, los ciclos, `readln` y recursividad. Lee un texto y un número.
`pila/`	Los tres	La prueba de la auxiliar: una pila con `Nodo.z` y `Pila.z`, manejada desde un menú.
`pruebas-p1/00_hello_world/`	Los tres	Hola mundo desde los tres lenguajes, y `Minimo.pig`.
`pruebas-p1/01_calculadora/`	PigLatin, Y?	Arreglos globales por referencia, un arreglo dentro de una estructura y recursividad.
`pruebas-p1/02_lista_doble/`	Los tres	Una lista doblemente enlazada: referencias encadenadas, `null`, sobrecarga y `elegir :.`
`pruebas-p1/03_laberinto/`	Los tres	Matrices como atributo con dimensiones variables, recursividad (flood fill) y `switch` con fallthrough.
`pruebas-p1/04_ahorcado/`	Los tres	Arreglos `char[]` y `boolean[]` creados con `new` en el constructor, `elegir :` y el ternario.
`p2/operacion/`	PigLatin, Zetariano	El ejemplo de clase y del parcial: el método recursivo `sumar`.
`p2/encapsulamiento/`	PigLatin, Zetariano	`public`, `protected`, default y `private` en atributos, métodos y constructores; default solo dentro de su clase. `errores.pig`: los 27 usos prohibidos.
`p2/herencia/`	PigLatin, Zetariano	`Animal` y `Perro` del enunciado: lo heredado, `super(...)`, el `super()` implícito y tres niveles de herencia. `errores.pig`: 7 errores de herencia.
`p2/polimorfismo/`	PigLatin, Zetariano	`Impresora`, `ImpresoraPDF` e `ImpresoraExcel` del enunciado en un arreglo: cada `imprimir()` responde por la clase del objeto. `errores.pig`: las reglas de `@Override`.
`p2/piglatin-objetos/`	PigLatin, Zetariano	`novus Profesor(..) : Persona` y `null : Persona` del enunciado. `errores.pig`: 4 errores de tipos de objeto.
`p2/integrador/`	Los tres	La misión de la resistencia: herencia en tres niveles, un arreglo de objetos, una matriz, recursividad en Y? y en Zetariano, y un arreglo que llega a Y? por referencia. Lee un nombre y un número.
`p2/banco/`	Los tres	El banco de pruebas de la auxiliar, con `movimientos` declarado `protected`: herencia, `@Override`, despacho sobre un arreglo de `Cuenta`, `null`, `double` con `int` y una estructura de Y? por referencia.

Mantenimiento, problemas y limitaciones

Dónde se cambia cada cosa.

Para cambiar...	Se toca
una palabra reservada o un símbolo	el lexer (`<Lenguaje>Lexer.g4`); el coloreado la toma solo, porque sale del lexer
una construcción del lenguaje	el parser (`.g4`), su constructor de AST (`*AstBuilder`) y, si es nueva, un nodo y su `visit` en `AstVisitor`
una regla semántica o un mensaje	`SemanticAnalyzer` (o `TypeSystem` si es de tipos)
el código intermedio	`ThreeAddressGenerator` y, si hay una cuarteta nueva, `ThreeAddressPrinter`
el ensamblador	`RiscVEmitter.translate`
la ejecución interna	`RiscVRunner` (por ejemplo `MAX_STEPS`)
la interfaz	`MainView` (controles) y `ApplicationController` (eventos)
los colores del editor	`src/main/resources/st4ck.css`

Agregar una palabra reservada

1. Declarar el token en el lexer, antes de `ID`: `NUEVA : 'nueva' ;`.
2. Usarlo en una regla del parser y reconstruir con `mvn -q clean compile`, que regenera el parser y su visitor.
3. Construir su nodo en el `*AstBuilder` del lenguaje, validarlo en `SemanticAnalyzer` y generarlo en `ThreeAddressGenerator`.
4. El coloreado es automático: `HighlightingCodeArea` pinta como palabra reservada todo token cuyo literal es una palabra, y como tipo los que están en su conjunto `TYPES`.

Agregar una cuarteta

1. Emitirla desde `ThreeAddressGenerator` con `emit(op, arg1, arg2, resultado)`.
2. Darle su forma de texto en `ThreeAddressPrinter`.
3. Traducirla en `RiscVEmitter.translate`. Una cuarteta sin traducción lanza `Cuarteta sin traduccion a RISC-V`.

Agregar un ejemplo

Se pone el `.pig` y sus importados en `examples/`, con su `<nombre>.entrada.txt` si lee algo y su `<nombre>.salida.txt` calculado a mano, y se corre la regresión. Un ejemplo de errores lleva en su cabecera cada error esperado.

Convenciones del código

- Código, comentarios y nombres en inglés; los mensajes al usuario, en español.
- Llaves al estilo Allman y asignaciones alineadas en el código nuevo. Lo existente no se reformatea.
- Principios KISS, DRY y YAGNI. Una simplificación con un límite conocido lleva un comentario `ponytail:` con el límite y cómo superarlo.
- Toda excepción se registra con su mensaje y su traza: `LOGGER.log(Level.SEVERE, mensaje, exception)`.

Solución de problemas

Problemas técnicos y su solución.

Síntoma	Causa	Solución
`Could not resolve dependencies ... rars:rars:jar:1.6`	Falta `lib/rars/rars/1.6/` en la copia del proyecto.	Clonar de nuevo o copiar la carpeta `lib/` completa; Maven toma el jar de ahí.
`NoClassDefFoundError: rars/...` al abrir la aplicación	Se ejecutó `App` sin el classpath de Maven.	Ejecutar con `mvn clean javafx:run` o desde el proyecto Maven del IDE.
`release version 21 not supported`	El JDK es anterior a 21.	Instalar un JDK 21 o superior y verificar con `java -version` y `mvn -v`.
Advertencias de `sun.misc.Unsafe` o acceso nativo al arrancar	JavaFX 21 sobre un JDK más nuevo.	Son inofensivas; el `pom.xml` ya pasa `--enable-native-access=javafx.graphics`.
`Filename too long` al clonar en Windows	Rutas de más de 260 caracteres.	`git -c core.longpaths=true clone ...`, o clonar en una carpeta más corta.
Acentos raros en la consola de RARS	La consola de Java no usa UTF-8.	Agregar `-Dstdout.encoding=UTF-8` al comando de RARS.
El comando de RARS falla en PowerShell (`<` no se admite, o `Could not find or load main class .encoding=UTF-8`)	PowerShell no redirige con `<` y parte `-Dstdout.encoding=UTF-8` en dos. Una tubería tampoco sirve: le antepone un BOM a la entrada.	Correrlo con `cmd /c "java -Dstdout.encoding=UTF-8 -jar ... main.asm < main.entrada.txt"`, o en Git Bash.

Síntoma	Causa	Solución
Un programa se detiene tras 11 millones de instrucciones	Un ciclo sin fin, o la entrada no tiene una línea para cada lectura.	Revisar la condición del ciclo, completar la "Entrada" o correrlo en RARS sin tope.
Errores de memoria en programas grandes	El heap es de 3 MiB (786 432 celdas) y no se libera.	Reducir lo que se reserva.

Limitaciones conocidas

- No hay chequeo de índices en tiempo de ejecución: el semántico solo valida los índices constantes.
- El heap es de 3 MiB y nunca se libera.
- Una llamada lleva a lo sumo 8 argumentos, contando el objeto de un método.
- Cada lectura consume una línea completa. Una cadena leída de la consola se asume ASCII, y `PrintChar` imprime el byte bajo: los caracteres fuera de Latin-1 no se representan.
- `ReadFloat` detiene el programa si la línea no es un número, a diferencia de la lectura de enteros.
- La ejecución interna recibe toda la entrada antes de empezar (no es interactiva), tiene el tope de 11 millones de instrucciones y corre un programa a la vez.
- Y? no tiene `++` prefijo. Además, una `}` sangrada al nivel de los `caso` y una línea que empieza con `(` justo después de una expresión se leen de otra forma.
- Solo un `.pig` se traduce y se ejecuta; un `.y` o un `.z` se analiza y ahí termina, porque no tiene `MAIOR>`.

Anexo: el ejercicio del parcial, paso a paso

El parcial pidió resolver a mano un ejercicio completo: una clase con un método recursivo, su código de tres direcciones, sus cuartetas y su RISC-V. El compilador hace los mismos pasos. Este anexo los muestra con la clase `Operacion` del parcial (`examples/p2/operacion`), y cada listado es la salida real del compilador.

El código fuente

La clase es la del parcial. Su `main` de Java pasa al `MAIOR>` del `.pig`, y por eso `id` y `valorBase` son `public`: sin modificador funcionarían como `private`.

```
public class Operacion {
    public int id;
    public int valorBase;

    public int sumar(int n) {
        if (n <= 0) {
            return this.valorBase;
        } else {
            int total = n + sumar(n - 1);
            return total;
        }
    }
}
```

```
import Operacion.z

VARIABILES>
esto obj : novus Operacion();
esto resultado : numerus 0;

MAIOR>
obj.id = 1;
obj.valorBase = 10;
resultado = obj.sumar(2);
>> "resultado = " >> resultado;
FINIS;
```

Imprime `resultado = 13`: `sumar(2)` es 2 + 1 + `valorBase`.

Código de tres direcciones

Así lo muestra la pestaña "Codigo 3D". El método recibe su objeto como primer parámetro (`this`), la condición se invierte para saltar al caso recursivo y el atributo se lee del heap:

```
func Operacion_sumar(this, n):
    if n > 0 goto L2
    t5 = this + 2
    t6 = heap[t5]
    return t6
    goto L1
L2:
    t7 = n - 1
    param this
    param t7
    t8 = call Operacion_sumar, 2
    t9 = n + t8
    total = t9
    return total
L1:
end_func Operacion_sumar
```

El `main` crea el objeto en el heap, guarda sus atributos y llama al método:

```
func main():
    t1 = H
    H = H + 3
    heap[t1] = 1
    param t1
    call Operacion_Operacion, 1
    obj = t1
    resultado = 0
    t2 = obj + 1
    heap[t2] = 1
    t3 = obj + 2
    heap[t3] = 10
    param obj
    param 2
    t4 = call Operacion_sumar, 2
    resultado = t4
    param "resultado = "
    call __print_str, 1
    print_int resultado
    print_char 10
    halt
end_func main
```

`heap[t1] = 1` guarda el `idClase` de `Operacion` en la celda 0 del objeto; `id` y `valorBase` quedan en las celdas 1 y 2. `__print_str` es la función auxiliar que imprime una cadena.

Cuartetas

Las mismas instrucciones como cuartetas, con el número que tienen en la pestaña "Cuartetas":

Cuartetas de Operacion_sumar.

#	op	arg1	arg2	resultado	C3D
26	func	this,n		Operacion_sumar	`func Operacion_sumar(this, n):`
27	if>	n	0	L2	`if n > 0 goto L2`
28	+	this	2	t5	`t5 = this + 2`
29	=[]	heap	t5	t6	`t6 = heap[t5]`
30	return	t6			`return t6`
31	goto			L1	`goto L1`
32	label			L2	`L2:`
33	-	n	1	t7	`t7 = n - 1`
34	param	this			`param this`
35	param	t7			`param t7`
36	call	Operacion_sumar	2	t8	`t8 = call Operacion_sumar, 2`
37	+	n	t8	t9	`t9 = n + t8`
38	=	t9		total	`total = t9`
39	return	total			`return total`
40	label			L1	`L1:`
41	end_func			Operacion_sumar	`end_func Operacion_sumar`

RISC-V

La traducción de `Operacion_sumar` en la pestaña "RISC-V". Cada bloque lleva encima, como comentario, la cuarteta de la que viene:

```
fn_Operacion_sumar:
    sw   ra, -4(sp)
    sw   fp, -8(sp)
    mv   fp, sp
    li   t6, 40
    sub  sp, sp, t6
    sw   a0, -12(fp)
    sw   a1, -16(fp)
    # if n > 0 goto L2
    lw   t0, -16(fp)
    li   t1, 0
    ble  t0, t1, _s1
    j    L2
_s1:
    # t5 = this + 2
    lw   t0, -12(fp)
    li   t1, 2
    add  t2, t0, t1
    sw   t2, -20(fp)
    # t6 = heap[t5]
    lw   t0, -20(fp)
    slli t0, t0, 2
    add  t0, t0, s11
    lw   t1, 0(t0)
```

```
    sw   t1, -24(fp)
    # return t6
    lw   a0, -24(fp)
    j    _exit_fn_Operacion_sumar
    # goto L1
    j    L1
L2:
    # t7 = n - 1
    lw   t0, -16(fp)
    li   t1, 1
    sub  t2, t0, t1
    sw   t2, -28(fp)
    # param this
    lw   a0, -12(fp)
    # param t7
    lw   a1, -28(fp)
    # t8 = call Operacion_sumar, 2
    jal  ra, fn_Operacion_sumar
    sw   a0, -32(fp)
    # t9 = n + t8
    lw   t0, -16(fp)
    lw   t1, -32(fp)
    add  t2, t0, t1
    sw   t2, -36(fp)
    # total = t9
    lw   t0, -36(fp)
    sw   t0, -40(fp)
    # return total
    lw   a0, -40(fp)
    j    _exit_fn_Operacion_sumar
L1:
    # end_func Operacion_sumar
_exit_fn_Operacion_sumar:
    lw   ra, -4(fp)
    lw   t0, -8(fp)
    mv   sp, fp
    mv   fp, t0
    ret
```

Equivalencias con la solución del parcial

La solución del parcial y el compilador hacen lo mismo, cada uno con su notación:

La solución del parcial y la salida del compilador.

Solución del parcial	**Este compilador**
`function sumar(obj, n):`	Un método recibe su objeto como primer parámetro, `this`: `func Operacion_sumar(this, n):`
`if n <= 0 goto et_base` y `goto et_recursivo`	`if n > 0 goto L2`: la condición se invierte y salta a la otra rama. Las etiquetas son `L1`, `L2`, …
`obj = malloc(8)`	`t1 = H` y `H = H + 3`: el objeto ocupa 3 celdas de 4 bytes, su `idClase` y sus dos atributos.
`mem[obj + 0] = 1`	`t2 = obj + 1` y `heap[t2] = 1`: `id` está en la celda 1.
`t_dir_base = obj + 4` y `t_val_base = mem[t_dir_base]`	`t5 = this + 2` y `t6 = heap[t5]`: `valorBase` está en la celda 2.
`total = n + t2`	`t9 = n + t8` y `total = t9`: cada operación pasa por un temporal.
`stack[sp + offset_ret] = total` y `goto et_salida`	`return total`: el valor viaja en `a0` y se salta a la única salida de la función.
Cuarteta `(9) call | sumar | 2 | t2`	Cuarteta `(36) call | Operacion_sumar | 2 | t8`.

Solución del parcial	Este compilador
Prólogo: `addi sp, sp, -32`, `sw ra, 24(sp)`, `sw a0, 20(sp)` y `sw a1, 16(sp)`	`sw ra, -4(sp)`, `sw fp, -8(sp)`, `mv fp, sp`, `sub sp, sp, t6` (40 bytes), `sw a0, -12(fp)` y `sw a1, -16(fp)`: el marco se recorre desde `fp`.
`ble a1, zero, et_base`	`ble t0, t1, _s1` y `j L2`: la rama salta sobre un `j`, que alcanza ±1 MiB.
`lw t0, 4(t2)`	`slli t0, t0, 2`, `add t0, t0, s11` y `lw t1, 0(t0)`: la celda pasa a dirección con la base del heap, `s11`.
`jal ra, sumar` y el resultado en `a0`	`jal ra, fn_Operacion_sumar` y `sw a0, -32(fp)`.
`et_salida:` y el epílogo: `lw ra, 24(sp)`, `addi sp, sp, 32` y `ret`	`_exit_fn_Operacion_sumar:` y `lw ra, -4(fp)`, `lw t0, -8(fp)`, `mv sp, fp`, `mv fp, t0` y `ret`.

En los dos casos la memoria se maneja a mano. El heap es el arreglo `heap[]` y `H` apunta a su siguiente celda libre: un `new` reserva moviendo `H`. La pila no aparece en el C3D: cada parámetro, variable y temporal tiene su ranura en el registro de activación, que el RISC-V recorre desde `fp`. Así cada llamada recursiva tiene su propio `n` y sus propios temporales, y `t9 = n + t8` usa el `n` de su llamada.